OrbitTrack

GESTÃO DE IMPRESSÃO 3D

DOCUMENTAÇÃO TÉCNICA

OrbitTrack Documentação técnica

Arquitetura, dados, segurança e roadmap do produto

18 capítulos | Auditoria real | Sem métricas inventadas

O que este documento cobre

- Requisitos, arquitetura e estrutura do código
- Base de dados, RLS, Storage e riscos
- Estado honesto de cada funcionalidade
- Roadmap, testes, deploy e anexos

Estado do produto

Versão de avaliação

Auditoria estática do código, sem validação em execução

Data	Versão
27 de outubro de 2026	1.0

Nota de rigor

Documento produzido a partir do código e do esquema SQL reais.
Sem clientes, métricas, preços ou certificações inventados.

CONTROLO DO DOCUMENTO

Documentação técnica

Arquitetura, dados, segurança e roadmap do produto

Campo	Detalhe
Tipo de documento	Documentação técnica de produto
Produto	OrbitTrack — Gestão de impressão 3D
Versão	1.0
Data	27 de outubro de 2026
Estado do produto	Versão de avaliação · nenhuma funcionalidade validada
Classificação	Confidencial · uso interno e avaliação
Autor	Equipa de produto OrbitTrack
Âmbito	Arquitetura, base de dados, segurança, módulos, deploy, testes, riscos e roadmap
Fontes	Repositório local, supabase/schema.sql, build, lint, tipos e npm audit
Revisão	1.0 · emissão inicial

HISTÓRICO DE REVISÕES

Versão	Data	Alteração	Responsável
1.0	27/10/2026	Emissão inicial para avaliação	Equipa de produto OrbitTrack

> **Nota de rigor**
>
> Documento produzido a partir do código e do esquema SQL reais. Não foram inventados clientes, métricas, preços, certificações ou resultados de testes. As funcionalidades planeadas estão identificadas como tal e as limitações não foram suavizadas.

ESTRUTURA

Documento organizado em 18 capítulos e anexos. Os capítulos 1 a 5 enquadram o produto e a experiência; os capítulos 6 a 12 cobrem módulos, dados e segurança; os capítulos 13 a 18 tratam qualidade, riscos, roadmap, operação e glossário.

NAVEGAÇÃO

Índice

Os números de página são atualizados no Word e correspondem ao documento final.

Resumo executivo 3
1. Visão geral 5
2. Requisitos do sistema 8
3. Arquitetura técnica 11
4. Estrutura do código 15
5. Interface e experiência do utilizador 19
6. Documentação funcional dos módulos 22
7. Regras de negócio e cálculos 29
8. Base de dados 32
9. Autenticação, autorização e segurança 38
10. Privacidade e conformidade 41
11. Instalação e ambiente de desenvolvimento 44
12. Testes e garantia de qualidade 47
13. Deploy, manutenção e operações 50
14. Desempenho e escalabilidade 53
15. Limitações, riscos e dívida técnica 55
16. Roadmap do produto 59
17. Operação e suporte 62
18. Glossário e anexos 65

Resumo executivo

A conclusão essencial em duas páginas: o que o produto é, o que está comprovado e o que impede a comercialização.

O OrbitTrack é uma aplicação web SaaS white-label para oficinas de impressão 3D. O vendedor gere encomendas num painel privado e o comprador acompanha o trabalho através de um link secreto, sem criar conta. A visão inclui ainda passaporte digital da peça, certificado de autenticidade verificável, recibos em PDF e QR codes.

Esta documentação foi produzida a partir de uma auditoria estática ao repositório local, ao esquema SQL e aos comandos de build, lint e tipos. Não houve acesso a um projeto Supabase em execução, pelo que nenhuma conclusão descreve comportamento observado em produção. As afirmações referem-se ao código existente e à sua leitura, não a resultados de testes.

O que existe de facto

- Estrutura React, Vite, TypeScript e Tailwind com rotas públicas e protegidas, contexto de autenticação e layout do painel.
- Ecrãs de login, registo, recuperação de palavra-passe, perfil de empresa, dashboard, lista de encomendas, criação de encomenda e configuração de estados.
- Esquema PostgreSQL com nove tabelas de negócio, RLS ativa, quatro RPCs públicas, dois buckets de Storage e funções de numeração.
- Páginas públicas de acompanhamento, passaporte e verificação de certificado, todas com problemas de ligação a dados.
- Geradores de PDF e visualizador 3D em esqueleto, sem fluxo completo de emissão ou download.

O que impede a comercialização

ID	Problema confirmado	Impacto direto
R-01	Build de produção bloqueado	Não é possível publicar na Vercel nem garantir tipos; o produto não tem artefacto de produção.
R-02	Páginas públicas bloqueadas pela RLS	O comprador não consegue acompanhar encomendas nem verificar certificados.
R-03	Criação de encomenda falha por numero_encomenda NOT NULL	O fluxo central do vendedor não funciona.
R-04	Buckets públicos expõem ficheiros de encomendas	Fotos, modelos 3D e documentos podem ser acedidos por qualquer pessoa com o URL.
R-05	Políticas de Storage demasiado permissivas	Um vendedor pode sobrescrever ou apagar ficheiros de outra empresa.
R-07	Sem testes de isolamento RLS	Não é possível afirmar que os dados estão isolados entre empresas.

Não apresentar como produto operacional

O build de produção falha, as páginas públicas não conseguem ler dados por RLS, a criação de encomendas viola uma coluna NOT NULL e não existe qualquer teste automatizado. Nenhuma funcionalidade atinge o estado “Implementada e validada”. A expressão “pronto para Vercel” não é sustentada pelo código atual.

Distribuição dos 40 itens auditados

Estado	Itens	Leitura correta
Implementada e validada	0	Nenhum item. Não existe suite de testes nem validação em execução.
Implementada, não validada	9	Código presente, mas sem prova de funcionamento ponta a ponta.
Parcialmente implementada	9	Existe uma parte do fluxo; faltam peças essenciais ou estão incompletas.
Bloqueada ou com problemas	11	O código existe, mas falha de forma confirmada ou impede o fluxo.
Planeada	10	Não existe implementação; apenas modelo, dependência ou intenção.
Não confirmada	1	Sem evidência suficiente para classificar.

A matriz completa, com evidência, limitações e próximo passo para cada item, está no Capítulo 6. Os riscos detalhados, com probabilidade, prioridade, esforço e critério de conclusão, estão no Capítulo 14 e no anexo de riscos.

Como usar este documento

- Decisores: ler o resumo executivo, o Capítulo 1, o Capítulo 14 e o roadmap do Capítulo 16.
- Equipa técnica: usar os Capítulos 3 a 12 como base de correção e os critérios de aceitação do Capítulo 15.
- Comercial: consultar o Capítulo 2 e o Capítulo 13 antes de qualquer proposta; não prometer fluxos bloqueados.
- Operação: seguir o Capítulo 17 para deploy e suporte, respeitando os bloqueios de segurança.

> **Convenção de estados**
>
> "Implementada e validada" exige código, ambiente real e teste reproduzível. "Implementada, não validada" significa que o código existe mas não foi provado. "Parcial" significa que só parte do fluxo existe. "Bloqueada" significa falha confirmada. "Planeada" significa que não há implementação.

1. Visão geral

O que é o OrbitTrack, para quem existe e qual é o estado real do produto nesta data.

1.1 Apresentação do produto

O OrbitTrack é uma aplicação web SaaS white-label para oficinas e estúdios de impressão 3D. Liga o vendedor ao comprador através de um painel privado de gestão e de uma página pública de acompanhamento acessível por um link secreto, sem criação de conta pelo comprador.

A visão funcional inclui três experiências complementares: o painel do vendedor, o portal de acompanhamento do comprador e o passaporte digital da peça, com certificado de autenticidade verificável. A implementação atual cobre apenas parte desta visão; o Capítulo 6 e a matriz de funcionalidades detalham o que existe, o que está bloqueado e o que é apenas planeado.

1.2 Problema que o produto pretende resolver

- Comunicação dispersa: o estado da encomenda vive em mensagens de WhatsApp, email e chamadas, obrigando a respostas repetidas.
- Processo invisível: o comprador não tem uma fonte única de verdade sobre o progresso, a previsão ou as alterações.
- Qualidade sem prova: depois da entrega não existe um registo verificável do material, dos parâmetros de impressão e da garantia.
- Marca diluída: pequenas oficinas competem com empresas maiores sem uma experiência digital que sustente um preço premium.

1.3 Público-alvo

Segmento	Necessidade principal	Relação com o produto
Estúdios de prototipagem	Reduzir suporte repetitivo e dar previsibilidade a clientes empresariais	Painel de encomendas e link público
Criadores de peças personalizadas	Valorizar o trabalho e comunicar proveniência	Passaporte digital e certificado
Pequenos fabricantes	Manter histórico de parâmetros e reposições	Ficha técnica, número de série e arquivo
Compradores finais	Saber o estado da encomenda sem criar conta	Página pública por token

1.4 Proposta de valor

- Um único link privado substitui dezenas de mensagens de acompanhamento.
- A experiência do comprador é mobile-first e não exige registo, palavra-passe ou aplicação.
- A oficina aplica a sua marca ao portal, aos documentos e ao certificado.
- A peça ganha identidade técnica e um código de verificação pública.
- A arquitetura multi-empresa foi desenhada desde o início, embora a eficácia do isolamento ainda não esteja validada por testes.

1.5 Âmbito atual e evolução prevista

Experiência	Estado nesta auditoria	Observação
Painel do vendedor	Parcial	Autenticação, perfil, estados e listagem existem. Criação de encomendas está bloqueada; edição e atualização de estado não estão implementadas.
Portal do comprador	Bloqueado	A interface existe, mas consulta tabelas protegidas como anon e falha por RLS. As RPCs públicas existem e não são usadas.
Passaporte da peça	Bloqueado	Página stub, parâmetro de rota incompatível e ausência de leitura de dados.
Certificados e recibos	Parcial	Modelo de dados e esqueletos de PDF. Não existe emissão, template ou fluxo de download.
QR codes	Planeado	Biblioteca instalada, sem qualquer utilização no código.

1.6 Método e limitações da auditoria

Esta documentação foi produzida a partir de uma análise estática do repositório local, da execução dos comandos de build e lint e da leitura integral do esquema SQL. Não foi possível validar o comportamento em execução contra um projeto Supabase real, porque não existe ambiente configurado nem credenciais no repositório.

- Fonte de verdade: ficheiros em src/, supabase/schema.sql, supabase/seed.sql, package.json e configurações do projeto.
- Comandos executados: npm run build, npx vite build, npm run lint, npx tsc -p tsconfig.json --noEmit --ignoreDeprecations 6.0 e npm audit.
- Não foram executadas migrações, seeds, alterações de código, commits ou testes de penetração.
- Não foram inventados clientes, métricas, preços, certificações, integrações ou resultados de testes.
- Sempre que não existe evidência suficiente, o estado é apresentado como "Não confirmada".

> **Conclusão de leitura rápida**
>
> Nenhuma funcionalidade de aplicação atinge o estado "Implementada e validada", porque não existe qualquer teste automatizado. Além disso, o build de produção falha, as páginas públicas estão bloqueadas pela RLS e a criação de encomendas falha por um campo obrigatório não preenchido. O produto não está pronto para comercialização.

1.7 Glossário essencial

Termo	Definição
White-label	Produto em que a marca visível é a do vendedor, não a do fornecedor da plataforma.
Token público	Cadeia aleatória longa usada no link do comprador; não é sequencial nem previsível.
RLS	Row Level Security: políticas do PostgreSQL que filtram linhas por utilizador/empresa.
RPC	Função remota do PostgreSQL exposta pelo Supabase e chamável pelo cliente.
Passaporte digital	Página pública com a identidade técnica e o histórico de uma peça.
Selo	Nível de qualidade atribuído pela oficina (bronze, prata ou ouro); não é uma certificação legal.

Termo	Definição
SLA	Acordo de nível de serviço; não existe nem está documentado no produto.
Tenant	Empresa isolada dentro da arquitetura multi-empresa.

1.8 Como ler os estados

Estado	Significado
Implementada e validada	Existe código e evidência de teste ou validação adequada.
Implementada, mas não validada	Existe código, mas não há testes ou confirmação suficiente.
Parcialmente implementada	Existem partes do fluxo, mas falta implementação ou integração.
Planeada	Está previsto, mas não existe implementação suficiente.
Bloqueada ou com problemas	Existe código ou dependência, mas há erro, bloqueio ou limitação conhecida.
Não confirmada	Não existe evidência suficiente para classificar.

1.9 Estrutura do documento

O documento segue dezoito capítulos. Os capítulos 2 a 9 constituem a referência técnica e funcional; os capítulos 10 a 14 cobrem conformidade, operação, qualidade e desempenho; os capítulos 15 a 18 consolidam riscos, roadmap, suporte e anexos. A matriz de funcionalidades do Anexo A é a fonte de verdade para qualquer afirmação sobre o estado do produto.

2. Requisitos do sistema

Requisitos funcionais, não funcionais e de segurança, com identificadores e estado de verificação.

2.1 Método de identificação

Os requisitos abaixo representam o âmbito necessário para operar o OrbitTrack como SaaS comercial. A coluna "Estado" não avalia a ambição do requisito: avalia apenas a evidência encontrada no código. Um requisito pode estar corretamente especificado e ainda assim não estar implementado.

2.2 Requisitos funcionais

ID	Requisito	Estado	Evidência e limitação principal
RF-001	Registar vendedor com email e palavra-passe	Implementada, mas não validada	Registar.tsx + Supabase Auth; sem testes e sem confirmação de email validada.
RF-002	Iniciar e terminar sessão	Implementada, mas não validada	Login.tsx, LayoutPainel.tsx e AuthContext; sem testes de sessão expirada.
RF-003	Recuperar palavra-passe	Bloqueada ou com problemas	Redirect para rota inexistente /painel/redefinir-senha.
RF-004	Criar empresa automaticamente no registo	Implementada, mas não validada	Trigger em auth.users; não validado e sem UNIQUE(utilizador_id).
RF-005	Editar perfil e identidade da empresa	Implementada, mas não validada	EmpresaPerfil.tsx; validação apenas HTML.
RF-006	Carregar logótipo e capa	Parcialmente implementada	Upload sem validação de tipo/tamanho; bucket público.
RF-007	Configurar estados por empresa	Implementada, mas não validada	EstadosConfig.tsx; sem unicidade e sem testes.
RF-008	Criar encomenda com dados do comprador	Bloqueada ou com problemas	Falta numero_encomenda NOT NULL; criação falha.
RF-009	Adicionar itens com ficha técnica	Parcialmente implementada	Formulário só expõe nome, quantidade e preço; restantes campos existem na base.
RF-010	Listar encomendas da empresa	Implementada, mas não validada	EncomendasLista.tsx; sem paginação, filtros ou ações.
RF-011	Editar encomenda existente	Bloqueada ou com problemas	fetchEncomenda vazio; não carrega nem atualiza.
RF-012	Atualizar estado com mensagem e fotos	Planeada	Tabelas existem; não existe UI nem handler.
RF-013	Registar histórico de estados	Parcialmente implementada	Tabela e RPC de leitura; nenhuma escrita pela aplicação.
RF-014	Gerar link público com token seguro	Parcialmente implementada	Token de 18 bytes no schema; não existe UI para copiar/partilhar.
RF-015	Acompanhar encomenda publicamente	Bloqueada ou com problemas	Página consulta tabelas como anon; RLS bloqueia.
RF-016	Mostrar timeline, estado e previsão	Parcialmente implementada	UI existe; sem dados e sem tratamento de previsão nula.
RF-017	Mostrar fotos no portal público	Planeada	Tabela e RPC devolvem fotos; UI não as apresenta.
RF-018	Mostrar ficha técnica e quantidade	Parcialmente implementada	UI mostra material, cor, camada e enchimento; faltam tempo, peso, impressora, série e data.
RF-019	Visualizar modelo 3D	Parcialmente implementada	Viewer3D apenas GLB/GLTF; STL não suportado; sem fallback.

ID	Requisito	Estado	Evidência e limitação principal
RF-020	Passaporte digital da peça	Bloqueada ou com problemas	Página stub e parâmetro de rota errado.
RF-021	Emitir recibo em PDF	Parcialmente implementada	Esboço em pdfGenerators.ts; sem itens, marca, numeração ou chamada.
RF-022	Emitir certificado em PDF	Parcialmente implementada	Esboço simples; sem selo gráfico, critérios, QR ou marca.
RF-023	Verificar certificado por código	Bloqueada ou com problemas	UI consulta tabela como anon, ignora :codigo e não apresenta todos os campos.
RF-024	Gerar QR PNG/PDF	Planeada	Biblioteca instalada; nenhuma utilização.
RF-025	Submeter avaliação pública	Planeada	RPC existe; não há formulário; sem validação de estado/abuso.
RF-026	Dashboard com indicadores	Parcialmente implementada	Contagem real; tempo médio e avaliação média fixos.
RF-027	Pesquisar, filtrar, ordenar e paginar	Planeada	Sem implementação; lista carrega tudo.
RF-028	Editor de template de certificado	Planeada	Não existe.
RF-029	Aplicar marca da empresa ao portal e documentos	Parcialmente implementada	Portal usa cor primária; documentos e certificado não.
RF-030	Contactar a oficina a partir do portal	Parcialmente implementada	Ícones de telefone e Instagram; sem email/WhatsApp nem formulário.

2.3 Requisitos não funcionais

ID	Requisito	Critério de aceitação	Estado
RNF-001	Desempenho percetível	Primeira renderização útil inferior a 2,5 s em 4G; lista paginada.	Não medido
RNF-002	Mobile-first	Todas as páginas utilizáveis entre 320 px e 430 px de largura.	Parcial
RNF-003	Acessibilidade	WCAG 2.2 AA nas páginas principais; navegação por teclado.	Não validado
RNF-004	Compatibilidade	Últimas duas versões de Chrome, Edge, Firefox e Safari.	Não validado
RNF-005	Disponibilidade	Objetivo a definir; sem SLA atual.	Não definido
RNF-006	Recuperação	RPO/RTO definidos; backups testados.	Não definido
RNF-007	Manutenibilidade	Build, lint e tipos verdes em CI.	Bloqueado
RNF-008	Observabilidade	Erros e eventos críticos registados com correlação.	Planeado
RNF-009	Privacidade	Minimização, retenção e direitos dos titulares documentados.	Planeado
RNF-010	Internacionalização	Textos externalizados; pt-PT como língua base.	Não implementado
RNF-011	Custos	Consumo previsível e alertas de faturação.	Não definido
RNF-012	Escalabilidade	Consultas paginadas e índices adequados a 10× o volume inicial.	Parcial

2.4 Requisitos de segurança

ID	Requisito	Evidência atual	Estado
SEG-001	RLS ativa em todas as tabelas de negócio	9 tabelas com RLS	Não validado
SEG-002	Isolamento entre empresas	Políticas por empresa_id/auth.uid()	Não validado
SEG-003	Acesso público apenas por RPC segura	RPCs existem, mas UI não as usa	Bloqueado
SEG-004	Buckets privados para dados de encomendas	public=true	Falha
SEG-005	Políticas de Storage por proprietário	Apenas bucket_id	Falha
SEG-006	Validação de uploads	Helpers existem, não são usados	Falha
SEG-007	Validação de entradas no servidor	Apenas constraints básicas	Parcial
SEG-008	Rate limiting em RPCs públicas	Não existe	Falha
SEG-009	Segredos fora do repositório	Apenas .env.example	Conforme
SEG-010	Sessões e recuperação seguras	Recuperação quebrada	Falha
SEG-011	Cabeçalhos de segurança e CSP	Não configurados	Planeado
SEG-012	Registo de auditoria	Apenas histórico de estados	Parcial

2.5 Restrições conhecidas

- Não existe backend próprio: toda a lógica de acesso passa pelo Supabase e pelas políticas RLS.
- O build de produção falha antes de gerar qualquer artefacto.
- Não existe ambiente de testes nem credenciais configuradas no repositório.
- A documentação não substitui revisão jurídica, fiscal ou de segurança.
- O produto não emite faturas certificadas; recibos e certificados são documentos da oficina, não documentos fiscais.

2.6 Critérios de aceitação para produção

01 Build e verificação de tipos terminam sem erros; existe pipeline de CI.
02 Todos os fluxos P0 (criar encomenda, link público, verificação) funcionam com dados reais.
03 Testes de RLS e Storage provam isolamento entre duas empresas.
04 Uploads validados, buckets privados e URLs assinadas em produção.
05 Recuperação de palavra-passe e confirmação de email testadas.
06 Documentação legal publicada e tratamento de dados revisto.
07 Monitorização, backups e procedimentos de incidente definidos.
08 Nenhuma métrica ou promessa comercial depende de valores fixos ou dados fictícios.

3. Arquitetura técnica

Como o OrbitTrack está construído, que serviços utiliza e onde estão as fronteiras de confiança.

3.1 Visão geral

O OrbitTrack é uma single-page application (SPA) React que comunica diretamente com o Supabase. Não existe servidor de aplicação próprio. O browser autentica-se com o Supabase Auth, recebe um JWT e usa o cliente Supabase para ler e escrever no PostgreSQL através do PostgREST. As políticas RLS decidem, linha a linha, o que cada utilizador pode ver ou alterar.

As páginas públicas usam a chave anónima e não têm sessão. A arquitetura prevê que essas páginas chamem funções RPC SECURITY DEFINER, que devolvem apenas o JSON necessário. Na implementação atual, as páginas públicas ainda consultam tabelas diretamente e, por isso, falham.

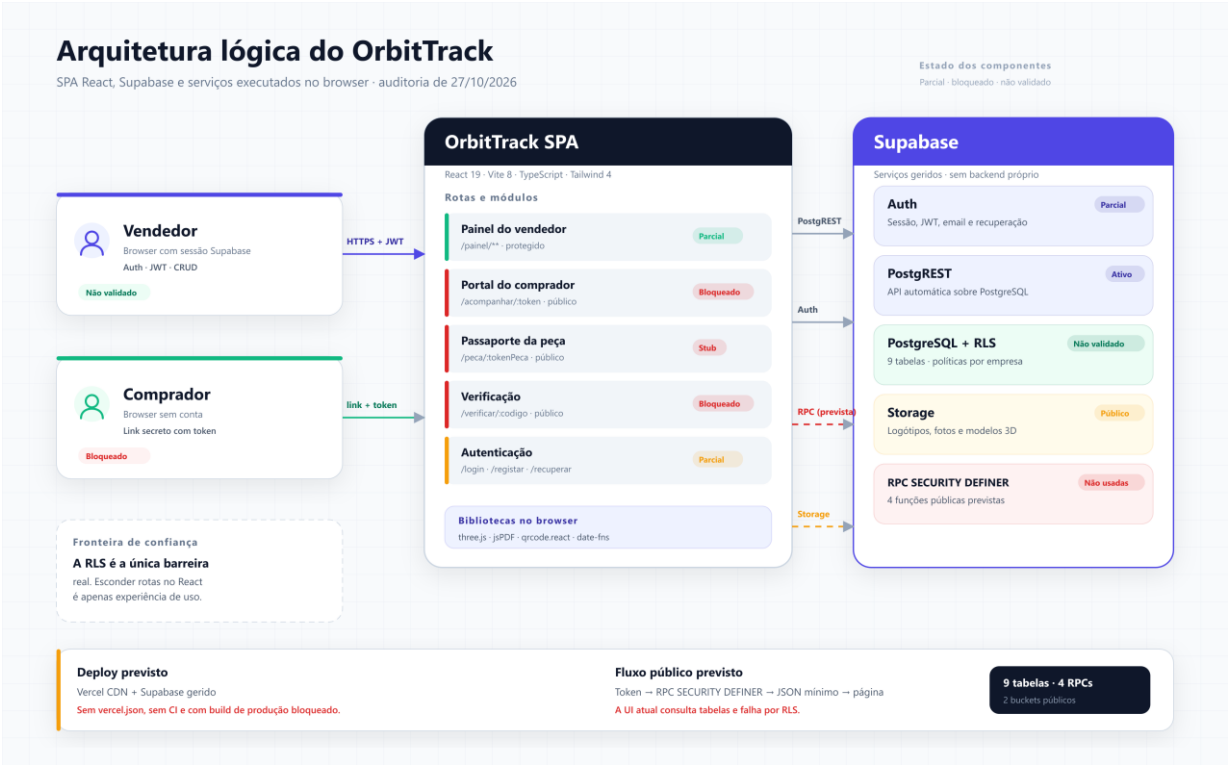


Figura 1 — Arquitetura lógica: SPA, Supabase e serviços executados no browser.

3.2 Tecnologias efetivamente utilizadas

Camada	Tecnologia	Versão no projeto	Responsabilidade
UI	React	19.3.1	Componentes, estado local e composição da interface.
Build	Vite	8.3.4 (instalado 8.3.5)	Servidor de desenvolvimento e bundling.
Linguagem	TypeScript	~6.0.3	Tipagem estática; verificação atualmente falha.
Estilos	Tailwind CSS	4.3.3	Utilitários e tema; build falha no @apply de classe personalizada.
Rotas	React Router	7.18.4	Navegação SPA e rotas protegidas.

Camada	Tecnologia	Versão no projeto	Responsabilidade
Backend	Supabase	@supabase/supabase-js 2.118.0	Auth, PostgREST, Storage e funções RPC.
Base de dados	PostgreSQL	Gerido pelo Supabase	Tabelas, índices, triggers e RLS.
3D	three.js + React Three Fiber + drei	0.186.1 / 9.8.2 / 10.7.9	Visualizador GLB/GLTF no browser.
PDF	jsPDF + jspdf-autotable	4.2.1 / 5.0.9	Geração de documentos no cliente (esboços).
QR	qrcode.react	4.0.1	Previsto para QR codes; não usado.
Datas	date-fns	4.4.0	Formatação de datas em pt-PT.
Ícones	lucide-react	0.468.0	Ícones da interface.
Documentação	pptxgenjs, docx, sharp, qrcode	Ferramentas de build	Geração dos materiais entregues; não fazem parte do runtime da aplicação.

3.3 Componentes e responsabilidades

- Browser do vendedor: executa a SPA, mantém a sessão Supabase e envia o JWT em cada pedido.
- Browser do comprador: executa a mesma SPA sem sessão; o token público é o único segredo de acesso.
- Supabase Auth: registo, login, sessão, recuperação e JWT.
- PostgREST: expõe tabelas e RPCs; aplica RLS e permissões por role.
- PostgreSQL: armazena empresas, encomendas, itens, histórico, fotos, recibos, certificados e avaliações.
- Storage: armazena logótipos, capas, fotos e modelos 3D; atualmente com buckets públicos.
- Serviços no browser: three.js para 3D, jsPDF para PDF e qrcode.react para QR (previsto).

3.4 Fluxo de dados do vendedor

01 O utilizador inicia sessão; o Supabase devolve uma sessão com JWT.
02 O AuthContext consulta a tabela empresas pelo utilizador autenticado.
03 As páginas do painel usam o cliente Supabase para SELECT/INSERT/UPDATE.
04 O PostgREST avalia as políticas RLS com auth.uid() e o empresa_id da linha.
05 Os dados autorizados são devolvidos ao browser e renderizados.

3.5 Fluxo de dados do comprador

01 O vendedor partilha um link com o token público (previsto; ainda sem UI).
02 O comprador abre /acompanhar/:token sem sessão.
03 A página deve chamar obter_encomenda_publica(token) com a chave anon.
04 A função SECURITY DEFINER valida o token e devolve apenas o JSON público.
05 A página renderiza estado, timeline, itens, fotos, recibo e certificado (previsto).

Estado real do fluxo público

A página actual não chama a RPC. Consulta encomendas, empresas, itens e histórico diretamente como anon. Como não existem políticas de SELECT para anon, a consulta é rejeitada e o comprador vê o estado de erro.

3.6 Autenticação e autorização

A autenticação é delegada no Supabase Auth. A autorização é implementada em duas camadas: o cliente esconde rotas e ecrãs, e o PostgreSQL aplica RLS. A segunda camada é a única que pode ser considerada uma barreira de segurança; a primeira é apenas experiência de utilização.

Área	Quem acede	Mecanismo	Estado
/login, /registar, /recuperar-senha	Público	Supabase Auth	Parcial
/painel/**	Vendedor autenticado	RotaProtegida + RLS	Não validado
/acompanhar/:token	Público com token	RPC prevista; atualmente tabela direta	Bloqueado
/peca/:tokenPeca	Público com token	RPC prevista; página stub	Bloqueado
/verificar/:codigo	Público com código	RPC prevista; atualmente tabela direta	Bloqueado

3.7 Storage

Bucket	Conteúdo	Visibilidade	Políticas atuais
empresas	Logótipos e fotos de capa	Público	Qualquer autenticado pode inserir/atualizar/apagar em qualquer caminho.
encomendas	Fotos, modelos 3D e ficheiros de encomendas	Público	Qualquer autenticado pode inserir/atualizar/apagar em qualquer caminho.

O modelo de caminho usado no upload é `${empresa.id}/${tipo}-${timestamp}.${ext}` para imagens de empresa. Não existe validação de que o primeiro segmento pertence ao utilizador autenticado. Esta é uma das falhas mais graves identificadas.

3.8 Geração de PDFs

A geração está prevista para o cliente, com jsPDF. As funções existentes em src/lib/pdfGenerators.ts são esboços: escrevem texto simples, não usam a marca da empresa, não incluem itens, selo, QR ou critérios e não são chamadas por nenhuma página. Não há geração no servidor nem assinatura digital.

3.9 Decisões de arquitetura

Decisão	Justificação	Consequência / estado
SPA + BaaS, sem backend próprio	Menor custo operacional e arranque rápido.	Segurança concentrada em RLS e RPC; exige testes rigorosos.
White-label por dados	Cada empresa define nome, cores, logótipo e contactos.	Não há instâncias separadas; o isolamento é lógico.
Token público longo	Acesso do comprador sem conta.	O link é um segredo; deve ser tratado como credencial.
RPC SECURITY DEFINER para leitura pública	Evita expor tabelas e políticas anon.	Existe no schema, mas não está ligada à UI.
PDF no cliente	Documentos não passam por servidores de terceiros.	Difícil garantir consistência, numeração e auditoria.

Decisão	Justificação	Consequência / estado
Deploy previsto na Vercel	SPA estática e CDN.	Falta vercel.json, CI e build funcional.

3.10 Topologia de deploy prevista

Elemento	Configuração atual	Necessário
Frontend	Sem vercel.json; build falha	Rewrites SPA, variáveis de ambiente e preview deployments.
Supabase	schema.sql manual; sem migrações versionadas	Migrações, ambiente de teste e backups.
Storage	Buckets públicos	Buckets privados e URLs assinadas.
Domínios	Não configurados	Domínio, TLS, redirect URLs e CORS.
CI/CD	Não existe	Pipeline de lint, tipos, testes e build.

> **Nota de arquitetura**
>
> A ausência de backend próprio não dispensa validação no servidor. Constraints, RLS, funções RPC e políticas de Storage são o "servidor" do OrbitTrack; qualquer regra que exista apenas no React pode ser contornada.

4. Estrutura do código

Mapa dos diretórios, rotas, componentes e convenções, com os caminhos confirmados no repositório.

4.1 Árvore de diretórios

```
orbittrack/
├─ public/
│  ├─ brand/                  # logótipo e variações (novos)
│  ├─ favicon.svg
│  └─ icons.svg
├─ src/
│  ├─ assets/                 # imagens de exemplo do template Vite
│  ├─ components/
│  │  ├─ painel/LayoutPainel.tsx
│  │  ├─ viewer3d/Viewer3D.tsx
│  │  └─ RotaProtegida.tsx
│  ├─ contexts/AuthContext.tsx
│  ├─ lib/
│  │  ├─ pdfGenerators.ts
│  │  ├─ supabase.ts
│  │  ├─ types.ts
│  │  └─ utils.ts
│  ├─ pages/
│  │  ├─ Login.tsx
│  │  ├─ Registar.tsx
│  │  ├─ RecuperarSenha.tsx
│  │  ├─ painel/{Dashboard,EncomendasLista,EncomendaForm,EmpresaPerfil,EstadosConfig}.tsx
│  │  └─ publico/{AcompanharEncomenda,PecaPassaporte,VerificarCertificado}.tsx
│  ├─ App.css
│  ├─ App.tsx
│  ├─ index.css
│  └─ main.tsx
├─ supabase/
│  ├─ schema.sql
│  └─ seed.sql
├─ scripts/                   # ferramentas de documentação e apresentação
├─ package.json
├─ tsconfig.json
└─ vite.config.ts
```

A árvore acima reflete os ficheiros existentes. Não foram encontrados testes, migrações versionadas, pipeline de CI, vercel.json ou documentação técnica prévia.

4.2 Rotas

Rota	Componente	Acesso	Estado
/login	Login.tsx	Público	Não validado
/registar	Registar.tsx	Público	Não validado
/recuperar-senha	RecuperarSenha.tsx	Público	Parcial (destino inexistente)
/acompanhar/:token	AcompanharEncomenda.tsx	Público	Bloqueado por RLS
/peca/:tokenPeca	PecaPassaporte.tsx	Público	Bloqueado/stub
/verificar/:codigo	VerificarCertificado.tsx	Público	Bloqueado por RLS
/verificar	VerificarCertificado.tsx	Público	Não validado
/painel	Dashboard.tsx	Protegido	Parcial
/painel/encomendas	EncomendasLista.tsx	Protegido	Não validado
/painel/encomendas/nova	EncomendaForm.tsx	Protegido	Bloqueado
/painel/encomendas/:id/editar	EncomendaForm.tsx	Protegido	Bloqueado/stub

Rota	Componente	Acesso	Estado
/painel/empresa	EmpresaPerfil.tsx	Protegido	Não validado
/painel/estados	EstadosConfig.tsx	Protegido	Não validado
*	Navigate para /login	Público	Não existe página 404

4.3 Responsabilidade dos ficheiros

Ficheiro	Responsabilidade	Observações
src/App.tsx	Define todas as rotas e o layout protegido.	Importação estática de todas as páginas; aumenta o bundle.
src/main.tsx	Monta React, importa o CSS e envolve a aplicação em AuthProvider.	AuthProvider duplicado: App também envolve AuthProvider.
src/contexts/AuthContext.tsx	Sessão, utilizador, empresa, login, registo, logout e recuperação.	Ignora erros de fetchEmpresa; redirect de recuperação inválido.
src/components/RotaProtegida.tsx	Redireciona para /login se não houver utilizador.	Proteção apenas client-side.
src/components/painel/LayoutPainel.tsx	Sidebar, navegação e logout.	Menu mobile funcional; botões sem aria-label.
src/pages/Login.tsx	Formulário de início de sessão.	Erro genérico; sem rate limiting visível.
src/pages/Registar.tsx	Formulário de registo.	Mínimo de 6 caracteres; navigate não usado.
src/pages/RecuperarSenha.tsx	Pedido de recuperação.	Não existe ecrã de redefinição.
src/pages/painel/Dashboard.tsx	Contagem de encomendas e dois cartões fixos.	Valores 3 dias e 4.8 não são calculados.
src/pages/painel/EncomendasLista.tsx	Tabela de encomendas.	Sem handlers, filtros ou paginação.
src/pages/painel/EncomendaForm.tsx	Criação de encomenda e itens.	Não envia numero_encomenda; edição vazia.
src/pages/painel/EmpresaPerfil.tsx	Perfil e uploads de marca.	Sem validação de ficheiros; pré-visualização simples.
src/pages/painel/EstadosConfig.tsx	CRUD de estados.	Reordenação local; gravação sequencial.
src/pages/publico/AcompanharEncomenda.tsx	Portal do comprador.	Consulta tabelas como anon; não usa RPC.
src/pages/publico/PecaPassaporte.tsx	Passaporte digital.	Stub; parâmetro errado.
src/pages/publico/VerificarCertificado.tsx	Verificação de certificado.	Consulta tabela; ignora :codigo.
src/components/viewer3d/Viewer3D.tsx	Visualizador 3D GLB/GLTF.	Sem STL, erro ou fallback.
src/lib/supabase.ts	Cliente Supabase e validação de env.	Lança erro se faltarem variáveis.
src/lib/types.ts	Tipos TypeScript do domínio.	Bom ponto de partida; alguns tipos duplicados nas páginas.
src/lib/utils.ts	Formatação, token, validação e limites.	Validadores não são usados; cn() não usa tailwind-merge.
src/lib/pdfGenerators.ts	Esboços de recibo e certificado.	Não usado; sem marca nem QR.
supabase/schema.sql	Tabelas, índices, funções, triggers, RLS e Storage.	Fonte de verdade da base de dados.
supabase/seed.sql	Dados de demonstração.	Inválido: FK e numero_encomenda em falta.

4.4 Gestão de estado

Não existe biblioteca de data fetching ou cache (React Query, SWR, Zustand). O estado é local a cada página, com useState/useEffect, e o contexto de autenticação guarda user, session e empresa. Isto é suficiente para um protótipo, mas origina pedidos repetidos, estados de erro inconsistentes e dificuldade em invalidar dados após mutações.

4.5 Validação de dados

- A validação no cliente resume-se a atributos HTML (required, type=email, minLength=6) e a algumas verificações manuais.
- Não existe schema de validação partilhado (Zod, Valibot ou equivalente).
- A validação no servidor depende de constraints SQL e de políticas RLS; não há validação semântica de cores, NIF, URLs ou estados.
- Os helpers validarImagem, validarFicheiro3D, TAMANHO_MAX_IMAGEM e TAMANHO_MAX_FICHEIRO existem em utils.ts, mas não são chamados.

4.6 Estilos e tema

O tema atual usa Tailwind CSS 4 com tokens em src/index.css. A cor primária da aplicação é #2563eb (azul), distinta da identidade indigo #4F46E5 usada nos materiais comerciais. O logótipo criado para a marca não está aplicado ao layout do painel, que continua a usar o ícone Printer e o texto “OrbitTrack”.

> **Inconsistência de marca**
>
> Os documentos e a apresentação usam a identidade OrbitTrack (indigo/esmeralda). A aplicação usa azul #2563eb e um ícone genérico. Antes de comercializar, alinhar tokens, logótipo, favicon e título da página.

4.7 Configuração e build

Ficheiro	Conteúdo	Observação
package.json	Scripts dev, build, lint e preview.	Não existe script test.
vite.config.ts	React, Tailwind e alias @.	Usa __dirname; aviso do novo config loader.
tsconfig.json	Strict, paths @/*, baseUrl.	baseUrl provoca TS5101 no TypeScript 6.
tsconfig.app.json	Configuração de app	Sem paths; falha a resolução de @/* se usada isoladamente.
index.html	Entrada Vite	Lang="en", título "orbittrack", sem meta description.
.env.example	URL, anon key e APP_URL	Valores placeholder; nunca inclui service role.
README.md	Instruções mínimas	Texto com caracteres corrompidos e sem estado real.

4.8 Estado do build e da qualidade

```
npm run build
> tsc -b && vite build
tsconfig.json(19,5): error TS5101: Option 'baseUrl' is deprecated...
```

```
npx vite build
Error: Cannot apply unknown utility class `btn`
src/index.css

npx tsc -p tsconfig.json --noEmit --ignoreDeprecations 6.0
src/lib/supabase.ts: Property 'env' does not exist on type 'ImportMeta'.
src/main.tsx: Cannot find module or type declarations for side-effect import of './index.css'.
src/pages/painel/EncomendaForm.tsx: Module 'lucide-react' has no exported member 'Cube'.
```

O lint (oxlint) termina com código 0, mas produz 40 avisos, incluindo imports não usados, dependências de hooks em falta e acesso a funções antes da inicialização. Não existe verificação de tipos verde nem build de produção.

4.9 Convenções e dívida técnica

- Nomenclatura mistura português e inglês: componentes e tabelas em português, tipos e APIs em inglês.
- Alguns componentes importam React sem o usar; outros usam any em vez dos tipos de src/lib/types.ts.
- Existem ficheiros mortos do template Vite (App.css, assets/react.svg, assets/vite.svg) e scripts antigos de geração.
- As páginas repetem tipos de domínio em vez de importar os tipos centrais.
- Não há barrel files nem separação clara entre serviços, hooks e componentes de apresentação.

5. Interface e experiência do utilizador

Fluxos reais de navegação, estados de interface e lacunas de usabilidade e acessibilidade.

5.1 Princípios visuais

- Layout limpo, cartões arredondados, sombras suaves e bastante espaço branco.
- Tipografia Inter com fallback de sistema; título de página forte e texto secundário em cinza.
- Formulários com labels visíveis, botões primários azuis e mensagens de sucesso/erro em cartões coloridos.
- Portal público com hero de capa, cartão de estado em destaque, timeline vertical e lista de peças.
- A experiência é mobile-first no portal do comprador; o painel usa sidebar colapsável em mobile.

5.2 Autenticação

Ecrã	Elementos	Estados	Lacunas
Login	Email, palavra-passe, mostrar/ocultar, links de recuperação e registo.	Loading, erro genérico.	Sem SSO, sem bloqueio por tentativas, sem mensagens específicas.
Registar	Email, palavra-passe, confirmação.	Validação de 6 caracteres, sucesso.	Não confirma política de palavra-passe forte; depende de email.
Recuperar	Email e confirmação de envio.	Loading, erro.	Destino do link não existe; fluxo incompleto.

5.3 Navegação do painel

A sidebar apresenta Dashboard, Encomendas, A Minha Empresa e Estados. Em mobile, abre por cima do conteúdo com overlay. O cabeçalho mobile mostra o nome OrbitTrack e o botão de menu. O rodapé da sidebar tem a ação Sair.

- A empresa ativa é apresentada sob o nome OrbitTrack, mas não há seletor de empresa porque cada utilizador tem uma empresa.
- Não existe breadcrumb, pesquisa global, notificações ou ajuda contextual.
- O botão de menu e os botões de ação da lista não têm aria-label.

5.4 Dashboard

O dashboard apresenta três cartões: total de encomendas, tempo médio de fabrico e avaliação média. Apenas o total é obtido por query. Os outros dois valores são fixos no código (“3 dias” e “4.8”), pelo que não devem ser apresentados como dados reais.

Dados não fiáveis

Os cartões de tempo médio e avaliação média mostram valores inventados. Em demonstração comercial, isto deve ser explicitamente identificado como exemplo ou removido até existir cálculo real.

5.5 Formulários

Formulário	Campos	Validação	Problema principal
Encomenda	Comprador, email, telefone, previsão, notas e itens.	HTML required; sem validação semântica.	Falta numero_encomenda; criação falha.
Empresa	Nome, NIF, descrição, cores, contactos, morada, rodapé.	HTML required no nome; email type.	Sem validação de NIF, URL, Instagram ou cor.
Estado	Nome, cor, mensagem.	Nome obrigatório.	Sem unicidade; ordem calculada no cliente.
Login/Registo	Email e palavra-passe.	HTML + mínimo 6.	Sem medidor de força ou política.

5.6 Listagens e tabelas

A lista de encomendas é uma tabela simples com número, comprador, estado, previsão e três botões de ação (editar, QR, documento). Nenhum dos botões tem handler. Não há pesquisa, filtros, ordenação, paginação, seleção múltipla ou estado vazio explícito. Em ecrãs pequenos, a tabela não tem contentor com scroll horizontal, podendo provocar overflow.

5.7 Modais e confirmações

O único diálogo de confirmação é o confirm() nativo do browser ao remover um estado. Não existem modais desenhados, drawers ou confirmações de operações destrutivas com contexto.

5.8 Estados de carregamento, vazio e erro

Ecrã	Carregamento	Vazio	Erro
Dashboard	Sem indicador visível.	Mostra 0.	Apenas console.error.
Lista de encomendas	Texto “A carregar...”.	Tabela vazia, sem mensagem.	Apenas console.error.
Perfil	Spinner no botão.	Formulário com valores vazios.	Cartão vermelho com mensagem.
Estados	Spinner.	“Nenhum estado configurado.”	Cartão vermelho.
Portal público	Spinner.	Não aplicável.	Mensagem “Ops!”, mas causada por RLS.
Verificação	Botão “A verificar...”.	Estado inicial sem instruções.	Cartão de código inválido.

5.9 Responsividade

- O painel adapta a sidebar e os formulários em grelha; o portal público foi desenhado para telemóvel.

- A tabela de encomendas e a pré-visualização de marca podem transbordar em ecrãs estreitos.
- Não foram feitos testes em dispositivos reais nem medições de toque.
- O visualizador 3D tem altura fixa de 400 px, sem adaptação a orientação.

5.10 Acessibilidade

Aspeto	Observação	Recomendação
Idioma	index.html tem lang="en".	Alterar para pt-PT.
Foco	Alguns botões têm focus:ring; outros não.	Garantir foco visível em todos os controlos.
Nomes acessíveis	Botões de ícone sem aria-label.	Adicionar nomes e estados.
Formulários	Labels associados por htmlFor na maioria.	Rever todos e adicionar descrições de erro.
Tabelas	Sem caption, scope ou resumo.	Adicionar semântica de tabela.
Contraste	Não medido.	Auditar com ferramenta e corrigir.
Movimento	Spinners e transições.	Respeitar prefers-reduced-motion.

5.11 Lacunas de experiência prioritárias

01 Dar feedback claro quando a criação de encomenda falha, sem alert() genérico.
02 Substituir as ações inertes da lista por botões funcionais ou removê-las.
03 Criar um estado vazio com chamada à ação na lista de encomendas.
04 Eliminar métricas fixas do dashboard ou identificá-las como exemplo.
05 Ligar o portal público às RPCs e desenhar o estado de link inválido.
06 Adicionar confirmação e undo para operações destrutivas.
07 Rever acessibilidade antes de qualquer demonstração com clientes.

6. Documentação funcional dos módulos

Finalidade, fluxo, regras, permissões e limitações de cada módulo identificado no código.

Cada ficha começa pelo estado de implementação e pela evidência principal. Quando um módulo não existe, é apresentado como planeado ou fora de âmbito, nunca como funcionalidade disponível.

6.1 Autenticação e conta

Campo	Detalhe
Estado	Implementada, mas não validada (login/registo); Bloqueada ou com problemas (recuperação)
Evidência	src/pages/Login.tsx, Registar.tsx, RecuperarSenha.tsx, src/contexts/AuthContext.tsx
Utilizadores	Vendedores; o comprador não tem conta.
Pré-condições	Projeto Supabase com Auth ativo, URL e anon key configurados; confirmação de email definida.
Dados	auth.users (Supabase) e empresas (perfil).

O registo cria um utilizador no Supabase Auth. O schema prevê um trigger AFTER INSERT em auth.users que cria automaticamente uma linha em empresas. O login usa signInWithPassword e guarda a sessão no cliente. O logout limpa o estado e redireciona para /login.

- A palavra-passe exige apenas 6 caracteres; não há política de complexidade, verificação de palavras-passe comprometidas ou MFA.
- A recuperação envia um email com redirectTo /painel/redefinir-senha, mas essa rota não existe em App.tsx. O fluxo termina num ecrã inexistente.
- O trigger de criação da empresa não foi testado; se falhar, o utilizador fica autenticado sem empresa e o perfil não funciona.
- Não existe ecrã de confirmação de email, reenvio ou alteração de palavra-passe dentro da aplicação.

6.2 Perfil da empresa e white-label

Campo	Detalhe
Estado	Parcialmente implementada
Evidência	src/pages/painel/EmpresaPerfil.tsx; tabela empresas; bucket empresas
Utilizadores	Vendedor autenticado, dono da empresa.
Pré-condições	Sessão ativa e registo em empresas.
Dados	Nome, descrição, logótipo, capa, cores, email, telefone, website, Instagram, morada, NIF e rodapé.

O formulário carrega os dados do contexto, permite editar e guardar via UPDATE em empresas. O upload de logótipo e capa envia o ficheiro para o bucket empresas e guarda o URL público no registo. Uma pré-visualização simples mostra nome, email, cor e um botão de exemplo.

- Não há validação de tipo, dimensão ou dimensões da imagem; qualquer ficheiro aceite pelo input image/* é enviado.
- O caminho inclui o id da empresa, mas a política de Storage não verifica se esse id pertence ao utilizador.

- NIF, website e Instagram são texto livre; não há normalização nem validação.
- A pré-visualização não replica o portal público nem os PDFs.
- O tema da aplicação não usa as cores da empresa; apenas o portal público as consome (quando funcionar).

6.3 Configuração de estados

Campo	Detalhe
Estado	Implementada, mas não validada
Evidência	src/pages/painel/EstadosConfig.tsx; tabela configuracao_estados; trigger criar_estados_padrao
Utilizadores	Vendedor autenticado.
Pré-condições	Empresa criada com os sete estados padrão.
Dados	Nome, ordem, cor e mensagem padrão.

A página lista os estados por ordem, permite editar nome/cor/mensagem, adicionar, remover e subir/descer. A reordenação é local até ao clique em Guardar Alterações, que atualiza cada linha sequencialmente.

- Não existe restrição UNIQUE para (empresa_id, nome) ou (empresa_id, ordem); são possíveis duplicados.
- A atualização é feita linha a linha, sem transação; uma falha a meio deixa o conjunto parcialmente atualizado.
- O estado da encomenda é texto livre; não existe chave estrangeira para configuracao_estados, pelo que renomear ou remover um estado não atualiza encomendas existentes.
- A mensagem padrão não é aplicada automaticamente porque não existe fluxo de alteração de estado.

6.4 Encomendas

Campo	Detalhe
Estado	Bloqueada ou com problemas (criação); Implementada, mas não validada (listagem); Bloqueada ou com problemas (edição)
Evidência	src/pages/painel/EncomendaForm.tsx, EncomendasLista.tsx; tabela encomendas
Utilizadores	Vendedor autenticado.
Pré-condições	Empresa existente e sessão ativa.
Dados	Comprador, contactos, estado, previsão, notas, token público e número.

O formulário de criação recolhe nome, email, telefone, previsão, notas e uma lista dinâmica de itens. A intenção é inserir a encomenda, obter o id e inserir os itens. A listagem mostra número, comprador, estado, previsão e ícones de ação.

> **Bloqueio confirmado**
>
> A tabela encomendas exige numero_encomenda NOT NULL, mas o formulário não o envia e a função gerar_numero_encomenda não é chamada. A inserção falha e o utilizador recebe um alert() genérico. Além disso, a encomenda e os itens são inseridos em duas operações sem transação; se a segunda falhar, fica uma encomenda sem itens.

- A edição está declarada na rota, mas fetchEncomenda está vazio; o formulário nunca carrega dados existentes.
- Não existem ações de eliminar, duplicar, arquivar ou exportar.
- A listagem não tem paginação, pesquisa, filtros ou ordenação.
- As notas internas nunca são apresentadas depois de guardadas.

6.5 Itens e ficha técnica

Campo	Detalhe
Estado	Parcialmente implementada
Evidência	src/pages/painel/EncomendaForm.tsx; tabela itens_encomenda
Utilizadores	Vendedor autenticado.
Pré-condições	Encomenda criada (atualmente bloqueada).
Dados	Nome, descrição, quantidade, preço, material, cor, camada, enchimento, tempo, peso, impressora, ficheiro 3D, série, token e data.

O schema suporta uma ficha técnica completa, mas o formulário só expõe nome, quantidade e preço unitário. Material, cor, camada e enchimento são predefinidos no estado inicial do item mas não têm campos editáveis. Os restantes campos só podem ser preenchidos por SQL manual.

- O preço unitário é numérico(10,2), mas o formulário aceita qualquer número e não calcula o total da encomenda.
- O token_peca é gerado por omissão (12 bytes, 24 caracteres hex), cumprindo o mínimo de 24 caracteres definido no âmbito.
- Não há upload de ficheiro 3D nem associação de fotos a um item.
- A quantidade não tem validação de mínimo ou inteiro positivo.

6.6 Atualização de estado e histórico

Campo	Detalhe
Estado	Planeada
Evidência	Tabelas encomendas.estado e historico_estados; nenhuma UI de alteração
Utilizadores	Vendedor autenticado (previsto).
Pré-condições	Encomenda existente.
Dados	Estado, mensagem, data/hora e fotos associadas.

O modelo prevê que cada alteração de estado crie uma linha em historico_estados com mensagem e timestamp, e que o portal público apresente a timeline. Não existe qualquer ecrã, botão ou função que faça essa escrita. Os ícones da lista de encomendas não têm handlers.

- Não há validação de transições (por exemplo, não permitir “Entregue” antes de “Enviada”).
- Não há associação entre a mensagem padrão configurada e a atualização efetiva.
- Não há notificação ao comprador; o link teria de ser consultado manualmente.
- O histórico é imutável na prática apenas por ausência de UI; a política RLS permite UPDATE/DELETE ao vendedor.

6.7 Fotos e ficheiros

Campo	Detalhe
Estado	Parcialmente implementada (modelo); Planeada (fluxo)
Evidência	Tabela fotos_encomenda; bucket encomendas; sem UI de upload de fotos de progresso
Utilizadores	Vendedor autenticado.
Pré-condições	Encomenda/item existente.
Dados	URL, legenda, item opcional e data.

A tabela permite fotos por encomenda e por item, com legenda. O portal público deveria mostrá-las, mas a UI atual não as renderiza. O único upload implementado é o de logótipo/capa da empresa; não existe upload de fotos de progresso nem de modelos 3D.

- O bucket encomendas é público; qualquer URL de fotografia pode ser acedido sem autenticação.
- Não há limites de tamanho, compressão, remoção de metadados EXIF ou validação de imagem.
- Não há associação entre a foto e o estado do histórico.

6.8 Portal público de acompanhamento

Campo	Detalhe
Estado	Bloqueada ou com problemas
Evidência	src/pages/publico/AcompanharEncomenda.tsx; RLS; RPC obter_encomenda_publica
Utilizadores	Comprador com link; não tem conta.
Pré-condições	Token público válido e encomenda existente.
Dados	Empresa, encomenda, itens, histórico, fotos, recibo, certificados, avaliação e estados.

A página implementa hero com capa e logótipo, cartão de estado, previsão, timeline, lista de peças e contactos. Visualmente está próxima do âmbito pretendido, mas a consulta é feita diretamente às tabelas com a chave anon. Como as políticas RLS só se aplicam a authenticated, a consulta é rejeitada e a página mostra “Não conseguimos encontrar esta encomenda”.

- A RPC obter_encomenda_publica já devolve o JSON necessário, mas não é usada.
- Não há botão para copiar o link, QR, download de recibo/certificado, formulário de avaliação ou botão de contacto por email/WhatsApp.
- A previsão de entrega nula é convertida em data inválida ou 01/01/1970.
- As fotos, o recibo e os certificados não são renderizados.
- A página não tem estado de “link expirado” nem explicação de privacidade.

6.9 Passaporte digital da peça

Campo	Detalhe
Estado	Bloqueada ou com problemas
Evidência	src/pages/publico/PecaPassaporte.tsx; rota /peca/:tokenPeca; RPC obter_peca_publica
Utilizadores	Proprietário da peça ou qualquer pessoa com o token (previsto).
Pré-condições	Token de peça válido.

Campo	Detalhe
Dados	Ficha técnica, empresa, encomenda, fotos, certificado e modelo 3D.

A página é um stub de 20 linhas. Lê token_peca do useParams, mas a rota define :tokenPeca, pelo que o valor é undefined. Não faz fetch, não mostra ficha técnica e passa a string "URL_DO_MODELO_3D_DA_PECA" ao visualizador.

- O modelo de dados e a RPC obter_peca_publica estão preparados.
- Não existe ligação a partir do portal público ou de um QR code.
- Não existe fallback quando a peça não tem modelo 3D.

6.10 Visualizador 3D

Campo	Detalhe
Estado	Parcialmente implementada
Evidência	src/components/viewer3d/Viewer3D.tsx
Utilizadores	Comprador (previsto) e vendedor (futuro).
Pré-condições	URL de modelo GLB/GLTF acessível.
Dados	Cena three.js com Stage, iluminação e OrbitControls.

O componente usa useGLTF e React Three Fiber. Não suporta STL nem GLTF com recursos externos mal empacotados. O Suspense tem fallback null (ecrã vazio durante o carregamento) e não existe error boundary; um URL inválido pode rebentar a árvore React.

- O âmbito prevê STL, GLB e GLTF; apenas GLB/GLTF são suportados.
- Não há controlo de qualidade, wireframe, medições ou comparação com o modelo original.
- O componente é importado estaticamente, aumentando o bundle inicial com three.js.
- Não há testes com modelos reais nem limites de tamanho.

6.11 Recibos

Campo	Detalhe
Estado	Parcialmente implementada
Evidência	Tabela recibos; gerar_numero_recibo; gerarReciboPDF em pdfGenerators.ts
Utilizadores	Vendedor (emissão); comprador (consulta prevista).
Pré-condições	Encomenda com valor e método de pagamento.
Dados	Número, data, valor total, método, observações e itens.

Existe tabela, função de numeração e um esboço de PDF. Não existe formulário de emissão, associação ao valor dos itens, download ou apresentação no portal. O PDF atual escreve apenas "RECIBO" e não lista itens.

> **Natureza do documento**
> Um recibo comercial da oficina não é uma fatura certificada. O OrbitTrack não deve ser apresentado como solução de faturação nem de conformidade fiscal sem validação legal e integração com software certificado.

6.12 Certificados e verificação

Campo	Detalhe
Estado	Parcialmente implementada (modelo/PDF); Bloqueada ou com problemas (verificação pública)
Evidência	Tabela certificados; RPC verificar_certificado; VerificarCertificado.tsx; pdfGenerators.ts
Utilizadores	Vendedor (emissão); público (verificação).
Pré-condições	Certificado emitido com código único.
Dados	Código, nível de selo, critérios, garantia, texto, empresa, peça e data.

O schema prevê níveis bronze, prata e ouro, critérios em JSON e garantia em meses. A RPC verificar_certificado normaliza maiúsculas e hífenes e devolve empresa, peça, série, nível, critérios, garantia, data e cor. A UI, porém, consulta a tabela diretamente como anon, procura o código em maiúsculas sem normalizar hífenes e ignora o parâmetro :codigo.

- Não existe ecrã de emissão, editor de critérios ou template por empresa.
- O código é gerado com RANDOM() e não com um CSPRNG.
- A verificação pública não mostra empresa, data ou nível de selo, apesar de o âmbito o exigir.
- O certificado é um selo de qualidade da oficina, não uma certificação legal.

6.13 QR codes

Campo	Detalhe
Estado	Planeada
Evidência	qrcode.react em package.json; nenhuma importação em src/
Utilizadores	Vendedor.
Pré-condições	Link público ou token de peça.
Dados	URL, imagem PNG e cartão PDF.

A biblioteca está instalada, mas não existe geração, download, cartão de embalagem ou impressão. O QR é uma peça central da proposta de valor (ligação físico-digital) e continua por implementar.

6.14 Avaliações

Campo	Detalhe
Estado	Planeada
Evidência	Tabela avaliacoes; RPC submeter_avaliacao
Utilizadores	Comprador com token; vendedor consulta (previsto).
Pré-condições	Encomenda existente e ainda não avaliada.
Dados	Estrelas (1–5), comentário e data.

A RPC verifica a existência da encomenda e impede duplicados, mas não valida o estado “Entregue”, não tem rate limiting e não existe formulário na página pública. O vendedor não vê avaliações no dashboard.

6.15 Dashboard

Campo	Detalhe
Estado	Parcialmente implementada
Evidência	src/pages/painel/Dashboard.tsx
Utilizadores	Vendedor autenticado.
Pré-condições	Sessão ativa.
Dados	Contagem total; valores fixos de tempo e avaliação.

A única métrica real é o número total de encomendas visíveis para o utilizador (a RLS filtra). O tempo médio e a avaliação média são strings fixas. Não há filtros por período, gráficos, comparação mensal, exportação ou estado vazio.

6.16 Módulos planeados e fora de âmbito

Módulo	Estado	Nota
Notificações automáticas por email/WhatsApp	Planeada	Fora do âmbito da versão atual; arquitetura não implementa filas nem provedores.
Pagamentos online	Planeada	Fora do âmbito; não há Stripe, MB Way ou integração bancária.
Câmara em direto	Planeada	Fora do âmbito; não há streaming.
Gestão de equipas e múltiplos utilizadores por empresa	Planeada	O modelo atual assume um utilizador por empresa.
Faturação certificada	Planeada	Requer software certificado e validação legal; não existe.
Editor de template de certificado	Planeada	Não existe.
Pesquisa, filtros e paginação	Planeada	Não existe.
Exportação de dados e eliminação de conta	Planeada	Não existe.

6.17 Leitura funcional

O OrbitTrack tem uma base de dados bem intencionada e uma interface com boa direção visual, mas os fluxos que justificam a compra ainda não estão fechados. O núcleo a corrigir é pequeno e identificável: build, numeração de encomendas, RPC pública, atualização de estado, PDFs/QR e testes de isolamento.

7. Regras de negócio e cálculos

Fórmulas, numeração, estados e tratamento de valores monetários encontrados no código.

7.1 Princípio

Só são documentadas regras efetivamente implementadas ou claramente previstas no schema. Quando uma regra não existe, é assinalada como lacuna. Não foram inventadas fórmulas fiscais, tabelas de preços ou métricas.

7.2 Numeração de encomendas

Aspeto	Implementação
Função	gerar_numero_encomenda(p_empresa_id UUID)
Formato	AAAA-NNNN (por exemplo, 2026-0001)
Cálculo	Ano atual + MAX(segundo segmento) + 1, filtrado por empresa e prefixo do ano.
Unidade	Texto; não é uma sequência PostgreSQL.
Concorrência	MAX+1 não é atómico; duas transações podem calcular o mesmo número.
Constraint	UNIQUE(empresa_id, numero_encomenda).
Estado	Função existe, mas não é chamada pela aplicação.

Exemplo: se a empresa tem 2026-0001 e 2026-0002, a função devolve 2026-0003. Se duas criações ocorrerem em simultâneo, ambas podem devolver 2026-0003; uma delas falhará na constraint.

7.3 Numeração de recibos

Aspeto	Implementação
Função	gerar_numero_recibo(p_empresa_id UUID)
Formato	REC-AAAA-NNNN
Cálculo	Ano + MAX(segundo segmento) + 1 entre recibos da empresa.
Constraint	Não existe UNIQUE em numero_recibo.
Estado	Função existe; não há fluxo de emissão.

7.4 Código de certificado

A função gerar_codigo_verificacao() constrói oito caracteres a partir do alfabeto ABCDEFGHJKLMNPQRSTUVWXYZ23456789 (sem caracteres ambíguos), com um hífen após o quarto caractere. Usa RANDOM(), não um gerador criptográfico. O código é único por constraint, mas não há tentativa automática em caso de colisão.

Propriedade	Valor
Formato	XXXX-XXXX
Alfabeto	32 caracteres

Propriedade	Valor
Comprimento	8 caracteres úteis
Entropia aproximada	40 bits (32^8)
Gerador	RANDOM() — não criptográfico
Estado	Implementado no schema; sem UI de emissão.

7.5 Tokens públicos

Token	Geração	Comprimento	Uso
token_publico	encode(gen_random_bytes(18), 'hex')	36 caracteres hex	Link de acompanhamento da encomenda.
token_peca	encode(gen_random_bytes(12), 'hex')	24 caracteres hex	Passaporte digital da peça.
codigo_verificacao	RANDOM() + alfabeto	9 caracteres com hífen	Verificação pública do certificado.

O token público de 18 bytes tem 144 bits de entropia, o que torna a adivinhação por força bruta inviável. O de peça tem 96 bits, também adequado. O código de certificado tem apenas 40 bits e é pesquisável publicamente; deve ser protegido por rate limiting.

7.6 Valores monetários

Os preços unitários e totais usam NUMERIC(10,2) no PostgreSQL. No frontend, os valores são convertidos para number de JavaScript, com o risco habitual de representação binária. A formatação usa Intl.NumberFormat pt-PT com EUR.

- O total da encomenda não é calculado nem persistido; o recibo tem valor_total independente.
- Não existe cálculo de IVA, descontos, portes ou retenções.
- Não existe política de arredondamento documentada; a base de dados arredonda a 2 casas ao gravar.
- O formulário aceita valores negativos, zero ou vazios sem validação de negócio.
- Não há soma de itens nem reconciliação entre encomenda, recibo e pagamento.

> **Exemplo ilustrativo**
>
> Uma encomenda com 2 peças a 42,45 € cada totaliza 84,90 €. Este total não é calculado pela aplicação; teria de ser introduzido manualmente no recibo. O exemplo serve apenas para explicar a fórmula esperada: quantidade × preço unitário.

7.7 Estado da encomenda

O estado é uma coluna TEXT com valor por omissão “Recebida”. A tabela configuracao_estados define nomes, ordem, cor e mensagem por empresa, mas não existe chave estrangeira nem validação de transição. A aplicação pode, em teoria, gravar qualquer string.

Regra	Estado
Estado inicial “Recebida”	Implementado por default.

Regra	Estado
Estados configuráveis por empresa	Implementado no schema e na UI.
Transições permitidas	Não definidas.
Atualização atómica com histórico	Não implementada.
Mensagem padrão automática	Não aplicada.
Notificação ao comprador	Não implementada.

7.8 Previsão de entrega

A previsão é uma DATE opcional. O formulário marca o campo como required, mas a coluna permite NULL. O portal público faz new Date(previsao_entrega).toLocaleDateString('pt-PT') sem verificar null, o que produz uma data indesejada. Não há cálculo automático de prazo, dias úteis ou capacidade da impressora.

7.9 Avaliações

A tabela impõe estrelas entre 1 e 5. A RPC submeter_avaliacao verifica a existência da encomenda pelo token e rejeita uma segunda avaliação. Não verifica se a encomenda foi entregue, não valida o comentário e não limita a frequência.

7.10 Regras de ficheiros

Tipo	Limite previsto em utils.ts	Aplicado?
Imagens	10 MB; .jpg, .jpeg, .png, .webp, .gif	Não
Modelos 3D	50 MB; .stl, .glb, .gltf	Não
Validação MIME	Não existe	Não
Dimensões de imagem	Não existe	Não

7.11 Casos-limite a testar

- Duas encomendas criadas no mesmo segundo para a mesma empresa.
- Preço unitário 0, negativo ou com mais de duas casas decimais.
- Encomenda sem itens ou com quantidade 0.
- Previsão de entrega nula ou anterior à data de criação.
- Token público inexistente, revogado ou com caracteres inválidos.
- Certificado com código em minúsculas, sem hífen ou com espaços.
- Duas avaliações simultâneas para a mesma encomenda.
- Upload de ficheiro com extensão permitida mas conteúdo diferente.
- Estado renomeado ou eliminado depois de usado em encomendas.

8. Base de dados

Esquema real, relações, índices, funções, triggers, RLS e limitações de integridade.

8.1 Visão geral

O esquema é composto por nove tabelas de negócio, dez funções, quatro triggers, oito índices explícitos e políticas RLS em todas as tabelas. A fonte de verdade é supabase/schema.sql. Não existem migrações versionadas; o ficheiro é aplicado manualmente no editor SQL do Supabase.

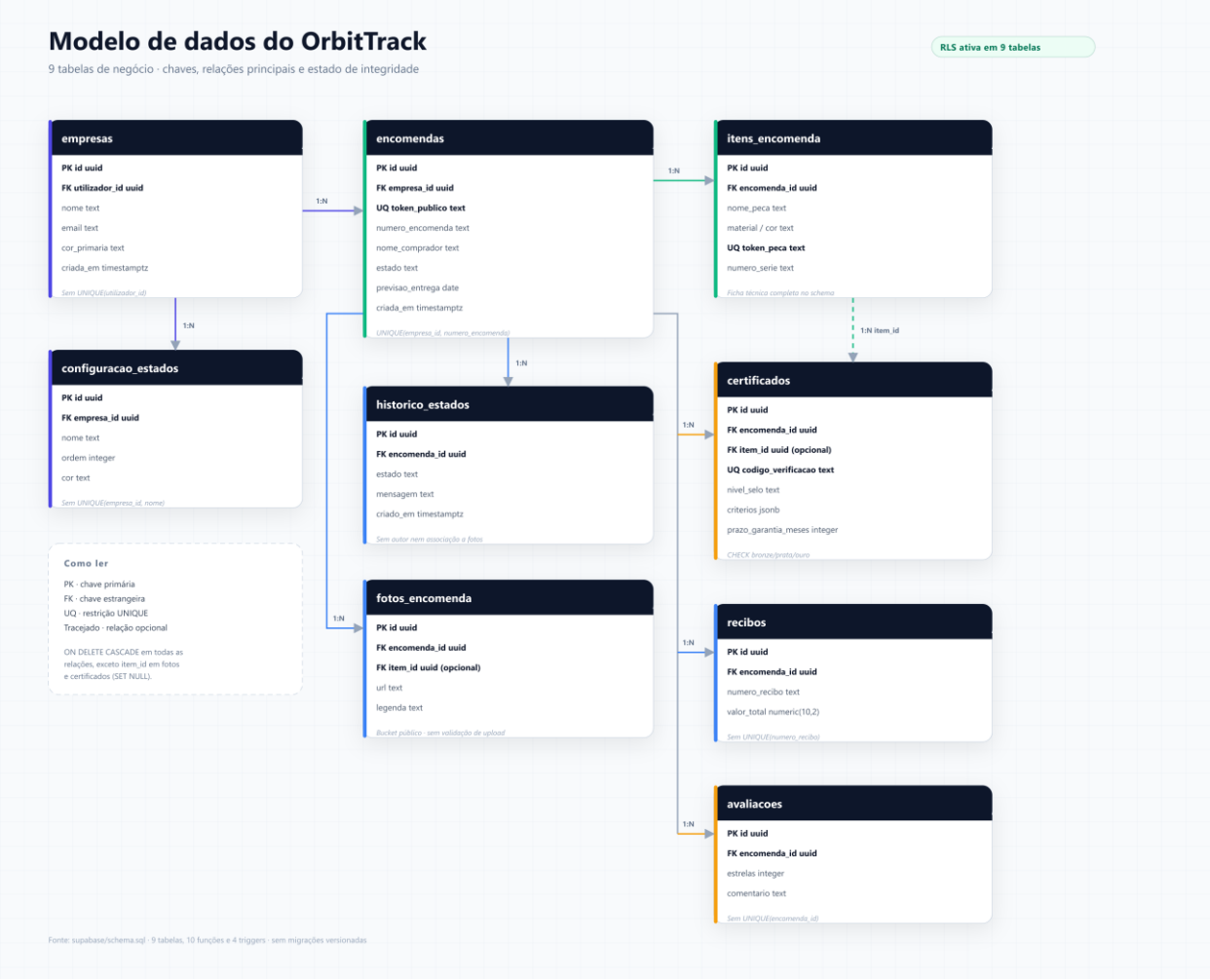


Figura 2 — Modelo entidade-relação simplificado do OrbitTrack.

8.2 Convenções

- Chaves primárias UUID com uuid_generate_v4().
- Timestamps com TIMESTAMPTZ e default NOW().
- Nomes de tabelas e colunas em português; funções em português.
- Valores monetários em NUMERIC(10,2).
- Tokens e códigos em TEXT.
- Não há schema dedicado; os objetos são criados no schema public por omissão.

8.3 Tabela empresas

Representa a oficina vendedora e a identidade white-label.

Coluna	Tipo	Nulo/default	Notas
id	UUID	PK, uuid_generate_v4()	Identificador da empresa.
utilizador_id	UUID	NOT NULL, FK auth.users(id) ON DELETE CASCADE	Dono; sem UNIQUE.
nome	TEXT	NOT NULL, default “A minha empresa”	Nome visível.
logotipo_url	TEXT	NULL	URL público do bucket empresas.
foto_capa_url	TEXT	NULL	URL público do bucket empresas.
cor_primaria	TEXT	NOT NULL, #2563eb	Cor do portal.
cor_secundaria	TEXT	NOT NULL, #1e40af	Cor secundária.
descricao	TEXT	NULL	Descrição/slogan.
email	TEXT	NULL	Contacto.
telefone	TEXT	NULL	Contacto.
website	TEXT	NULL	URL.
instagram	TEXT	NULL	Handle.
morada	TEXT	NULL	Morada.
nif	TEXT	NULL	NIF; sem validação.
texto_rodape	TEXT	Default “Obrigado pela sua encomenda!”	Rodapé.
criada_em	TIMESTAMPTZ	NOT NULL, NOW()	Criação.
atualizada_em	TIMESTAMPTZ	NOT NULL, NOW()	Atualizada por trigger.

8.4 Tabela configuracao_estados

Coluna	Tipo	Nulo/default	Notas
id	UUID	PK	Identificador.
empresa_id	UUID	NOT NULL, FK empresas ON DELETE CASCADE	Dono do estado.
nome	TEXT	NOT NULL	Nome apresentado.
ordem	INTEGER	NOT NULL, default 0	Ordenação; sem UNIQUE.
cor	TEXT	NOT NULL, #6b7280	Cor do estado.
mensagem_padrao	TEXT	NULL	Mensagem sugerida.
criado_em	TIMESTAMPTZ	NOT NULL, NOW()	Criação.

8.5 Tabela encomendas

Coluna	Tipo	Nulo/default	Notas
id	UUID	PK	Identificador.
empresa_id	UUID	NOT NULL, FK empresas CASCADE	Dono.
token_publico	TEXT	NOT NULL, UNIQUE, gen_random_bytes(18) hex	36 caracteres.
numero_encomenda	TEXT	NOT NULL	Formato AAAA-NNNN; obrigatório.
nome_comprador	TEXT	NOT NULL	Dado pessoal.

Coluna	Tipo	Nulo/default	Notas
email_comprador	TEXT	NULL	Dado pessoal.
telefone_comprador	TEXT	NULL	Dado pessoal.
estado	TEXT	NOT NULL, default “Recebida”	Texto livre.
previsao_entrega	DATE	NULL	Data prevista.
notas_publicas	TEXT	NULL	Visíveis no portal.
notas_internas	TEXT	NULL	Apenas vendedor.
criada_em	TIMESTAMPTZ	NOT NULL, NOW()	Criação.
atualizada_em	TIMESTAMPTZ	NOT NULL, NOW()	Trigger.

Restrições: UNIQUE(empresa_id, numero_encomenda); UNIQUE(token_publico). Índices: idx_encomendas_empresa, idx_encomendas_token.

8.6 Tabela itens_encomenda

Coluna	Tipo	Nulo/default	Notas
id	UUID	PK	Identificador.
encomenda_id	UUID	NOT NULL, FK encomendas CASCADE	Dono.
nome_peca	TEXT	NOT NULL	Nome.
descricao	TEXT	NULL	Descrição.
quantidade	INTEGER	NOT NULL, default 1	Sem CHECK > 0.
preco_unitario	NUMERIC(10,2)	NULL	Preço.
material	TEXT	NULL	PLA, PETG, etc.
cor	TEXT	NULL	Cor.
altura_camada	TEXT	NULL	Texto, não numérico.
enchimento_percentagem	INTEGER	NULL	Sem CHECK 0–100.
tempo_impressao_minutos	INTEGER	NULL	Duração.
peso_gramas	NUMERIC(10,2)	NULL	Peso.
impressora_usada	TEXT	NULL	Equipamento.
ficheiro_3d_url	TEXT	NULL	URL pública.
numero_serie	TEXT	NULL	Série.
token_peca	TEXT	UNIQUE, gen_random_bytes(12) hex	24 caracteres.
data_impressao	DATE	NULL	Data.
criado_em	TIMESTAMPTZ	NOT NULL, NOW()	Criação.

8.7 Tabelas de histórico, fotos, recibos, certificados e avaliações

Tabela	Colunas principais	Relações e notas
historico_estados	id, encomenda_id, estado, mensagem, criado_em	FK encomendas CASCADE; não guarda autor nem fotos.
fotos_encomenda	id, encomenda_id, item_id, url, legenda, criada_em	FK encomendas CASCADE; FK itens SET NULL.

Tabela	Colunas principais	Relações e notas
recibos	id, encomenda_id, numero_recibo, data_emissao, valor_total, metodo_pagamento, observacoes, criado_em	FK encomendas CASCADE; numero_recibo sem UNIQUE.
certificados	id, encomenda_id, item_id, codigo_verificacao, texto_personalizado, nivel_selo, criterios, prazo_garantia_meses, emitido_em	FK encomendas CASCADE; FK itens SET NULL; CHECK bronze/prata/ouro; UNIQUE código.
avaliacoes	id, encomenda_id, estrelas, comentario, criada_em	FK encomendas CASCADE; CHECK 1–5; sem UNIQUE(encomenda_id).

A ausência de UNIQUE(encomenda_id) em avaliacoes é compensada pela verificação na RPC submeter_avaliacao, mas não por uma constraint; duas chamadas concorrentes podem, em teoria, inserir duas avaliações.

8.8 Índices

Índice	Tabela/coluna	Objetivo
idx_encomendas_empresa	encomendas(empresa_id)	Listagem por empresa.
idx_encomendas_token	encomendas(token_publico)	Acesso público.
idx_itens_encomenda	itens_encomenda(encomenda_id)	Itens da encomenda.
idx_itens_token_peca	itens_encomenda(token_peca)	Passaporte.
idx_historico_encomenda	historico_estados(encomenda_id)	Timeline.
idx_fotos_encomenda	fotos_encomenda(encomenda_id)	Galeria.
idx_certificados_codigo	certificados(codigo_verificacao)	Verificação.
idx_configuracao_empresa	configuracao_estados(empresa_id)	Estados por empresa.

Os índices UNIQUE de token_publico, token_peca e codigo_verificacao criam índices implícitos adicionais. Não existem índices compostos para listagens ordenadas por criada_em nem para pesquisa textual.

8.9 Funções e triggers

Objeto	Tipo	Responsabilidade	Estado
gerar_numero_encomenda	Função	Número AAAA-NNNN por empresa.	Não usada
gerar_numero_recibo	Função	Número REC-AAAA-NNNN por empresa.	Não usada
gerar_codigo_verificacao	Função	Código XXXX-XXXX com RANDOM().	Não usada
atualizar_timestamp	Função + 2 triggers	Atualiza atualizada_em em empresas/encomendas.	Ativa
criar_estados_padrao	Função + trigger	Cria 7 estados após INSERT em empresas.	Ativa
criar_empresa_para_utilizador	Função + trigger	Cria empresa após INSERT em auth.users.	Não validada
obter_encomenda_publica	RPC	Devolve JSON público por token.	Não usada
obter_peca_publica	RPC	Devolve JSON da peça por token.	Não usada
verificar_certificado	RPC	Valida código e devolve dados.	Não usada
submeter_avaliacao	RPC	Insere avaliação única.	Não usada

8.10 Row Level Security

Tabela	SELECT	INSERT	UPDATE	DELETE
empresas	Dono	Sem política (trigger)	Dono	Sem política
configuracao_estados	Dono	Dono	Dono	Dono
encomendas	Dono	Dono	Dono	Dono
itens_encomenda	Dono	Dono	Dono	Dono
historico_estados	Dono	Dono	Dono	Dono
fotos_encomenda	Dono	Dono	Dono	Dono
recibos	Dono	Dono	Dono	Dono
certificados	Dono	Dono	Dono	Dono
avaliacoes	Dono	Sem política (RPC)	Sem política	Sem política
anon	Nenhum acesso direto	Nenhum	Nenhum	Nenhum

"Dono" significa que a linha pertence a uma empresa cujo utilizador_id = auth.uid(). As políticas de avaliações permitem apenas SELECT ao vendedor; a inserção é feita pela RPC SECURITY DEFINER. Não existem políticas para anon em nenhuma tabela.

8.11 Storage

Bucket	public	Políticas	Risco
empresas	true	INSERT/UPDATE/DELETE para authenticated; SELECT public	Qualquer autenticado pode alterar qualquer caminho.
encomendas	true	INSERT/UPDATE/DELETE para authenticated; SELECT public	Ficheiros de encomendas acessíveis por URL e alteráveis por qualquer autenticado.

8.12 Migrações e idempotência

Não existe pasta de migrações nem histórico de versões. schema.sql cria tabelas sem IF NOT EXISTS e políticas sem verificação de existência; uma segunda execução falha. O seed também não é idempotente e não pode ser executado como está.

> **Dependência não declarada**
>
> O schema usa gen_random_bytes(), mas só declara CREATE EXTENSION IF NOT EXISTS "uuid-ossp". No Supabase, a extensão pgcrypto costuma estar disponível, mas o schema deve declará-la explicitamente. A função também deveria fixar o search_path.

8.13 Integridade e lacunas

- Falta UNIQUE(utilizador_id) em empresas; o trigger pode criar duplicados em cenários de reprocessamento.
- Falta UNIQUE(empresa_id, nome) e UNIQUE(empresa_id, ordem) em configuracao_estados.

- Falta FK de encomendas.estado para configuracao_estados; renomear estados não propaga.
- Falta UNIQUE(numero_recibo) por empresa e uma constraint de formato.
- Falta UNIQUE(encomenda_id) em avaliacoes.
- Faltam CHECKs para quantidade > 0, enchimento 0–100, preço >= 0 e NIF.
- Falta ON DELETE para utilizador em empresas além do CASCADE de auth.users; apagar o utilizador elimina a empresa e os dados associados.
- Não há auditoria de alterações nem soft delete.

9. Autenticação, autorização e segurança

Modelo de acesso, RLS, RPCs, Storage e riscos de segurança com prioridade.

9.1 Autenticação

A autenticação é gerida pelo Supabase Auth com email e palavra-passe. O cliente guarda a sessão e o JWT; o AuthContext expõe user, session e empresa. O registo pode exigir confirmação de email, dependendo da configuração do projeto Supabase, que não está documentada.

- Não existe MFA, SSO ou login social.
- A palavra-passe mínima tem 6 caracteres; não há verificação de palavras-passe comprometidas.
- O logout limpa o estado local e a sessão Supabase.
- A recuperação de palavra-passe está quebrada por falta de rota de destino.

9.2 Modelo de autorização

Cada utilizador autenticado está associado a uma empresa através de empresas.utilizador_id. As políticas RLS derivam o empresa_id a partir dessa associação. Não existem papéis (admin, operador, leitor) nem membros de equipa.

Perfil	Acesso	Mecanismo
Anon (comprador)	Apenas RPCs públicas e Storage público	Chave anon; sem sessão.
Authenticated (vendedor)	Dados da própria empresa	JWT + RLS.
service_role	Acesso total	Nunca deve estar no cliente; não é usado no código.

9.3 RLS: o que está provado

As políticas existem e estão ativas. O que não existe é evidência de que funcionam como esperado em todos os caminhos. Não foi executado qualquer teste com dois utilizadores, nem testes de INSERT/UPDATE/DELETE cruzados.

- As políticas de empresas permitem SELECT/UPDATE ao dono; não há INSERT/DELETE para authenticated.
- As políticas de configuração, encomendas, itens, histórico, fotos, recibos e certificados usam subqueries para empresas e permitem ALL ao dono.
- A política de avaliações permite apenas SELECT ao dono; a escrita é feita por RPC.
- Não há políticas para anon, pelo que qualquer SELECT direto das páginas públicas falha.
- As políticas de Storage não verificam o proprietário do caminho.

9.4 Funções RPC públicas

Função	Entrada	Devolve	Riscos
obter_encomenda_publica	token	Empresa, encomenda, itens, histórico, fotos, recibo, certificados, avaliação e estados	Sem search_path fixo; devolve IDs internos; sem rate limit.
obter_peca_publica	token_peca	Peça, empresa, encomenda, fotos e certificado	Idem; devolve ficheiro_3d_url público.

Função	Entrada	Devolve	Riscos
verificar_certificado	código	Validade e dados do certificado	Idem; código de 40 bits enumerável.
submeter_avaliacao	token, estrelas, comentário	Sucesso/erro	Sem validação de estado entregue; sem rate limit.

As quatro funções são SECURITY DEFINER, ou seja, executam com os privilégios do criador e ignoram a RLS das tabelas subjacentes. Isto é necessário para o acesso público, mas exige validação rigorosa de entradas, limites de resposta e fixação do search_path.

```
-- Recomendação de endurecimento (exemplo, não aplicado)
CREATE OR REPLACE FUNCTION public.obter_encomenda_publica(p_token TEXT)
RETURNS JSON
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public, pg_temp
AS $$
BEGIN
  IF p_token IS NULL OR length(p_token) < 24 THEN
    RETURN NULL;
  END IF;
  -- ... consulta ...
END;
$$;
```

9.5 Storage

Os dois buckets são públicos. As políticas de INSERT/UPDATE/DELETE permitem a qualquer utilizador autenticado operar sobre qualquer objeto do bucket, desde que o bucket_id corresponda. Não existe verificação de que o caminho começa pelo empresa_id do utilizador.

> **Falha crítica de isolamento**
>
> Um vendedor autenticado pode tentar sobrescrever ou apagar ficheiros de outra empresa se conhecer o caminho. Como os URLs são públicos, também pode ler ficheiros de encomendas sem autenticação. Isto contradiz a promessa de isolamento multi-empresa.

9.6 Validação no cliente e no servidor

- O cliente usa validação HTML superficial e não valida ficheiros.
- O servidor depende de constraints e RLS; não há validação semântica centralizada.
- Não há sanitização de HTML, porque o React escapa conteúdo por omissão; não foram encontrados dangerouslySetInnerHTML.
- Não há limites de tamanho de pedido, paginação ou quotas por empresa.

9.7 Segredos e configuração

O repositório contém apenas .env.example com placeholders. A chave anon é pública por desenho e pode estar no bundle; a service_role nunca deve ser usada no cliente. Não foi encontrado ficheiro .env com credenciais.

- Não há rotação de chaves documentada.
- Não há configuração de redirect URLs, CORS, CAPTCHA ou limites de Auth.

- Não há cabeçalhos de segurança (CSP, HSTS, X-Frame-Options) configurados.
- Não há logs de autenticação ou alertas de abuso.

9.8 Matriz de riscos de segurança

Risco	Gravidade	Probabilidade	Mitigação prioritária
Buckets públicos expõem dados	Crítica	Confirmada	Tornar privado; URLs assinadas.
Políticas de Storage sem dono	Crítica	Confirmada	Restringir por empresa_id/auth.uid().
Sem testes RLS	Crítica	Alta	Suite com dois utilizadores.
Páginas públicas quebradas	Alta	Confirmada	Ligar às RPCs.
RPCs sem search_path	Alta	Média	SET search_path.
Sem rate limiting	Média	Alta	Limites no edge/funções.
Código de certificado fraco	Média	Média	CSPRNG + retry.
Recuperação de conta quebrada	Alta	Confirmada	Implementar rota.
Sem MFA	Média	Média	Oferecer TOTP.
Sem logs de auditoria	Média	Alta	Registar eventos críticos.

9.9 Testes de segurança necessários

01 Utilizador A tenta SELECT de encomendas da empresa B: deve devolver zero linhas.
02 Utilizador A tenta UPDATE/DELETE de uma encomenda de B: deve falhar.
03 Utilizador A tenta INSERT de uma encomenda com empresa_id de B: deve falhar.
04 Anon tenta SELECT direto de todas as tabelas: deve falhar.
05 Anon chama as RPCs com token válido, inválido e vazio: deve obter respostas controladas.
06 Utilizador A tenta upload/download/delete no caminho de B: deve falhar.
07 Token público com 23 caracteres e com caracteres inválidos: deve ser rejeitado.
08 Repetição massiva de verificação de certificados: deve ser limitada.

9.10 Veredicto

Não é possível declarar o sistema seguro

A existência de RLS e de autenticação é um bom ponto de partida, mas não é prova de segurança. Sem testes de isolamento, com buckets públicos e com políticas de Storage permissivas, o OrbitTrack não deve ser apresentado como multi-empresa seguro nem disponibilizado comercialmente no estado atual.

10. Privacidade e conformidade

Dados pessoais tratados, obrigações a validar e limites das afirmações de conformidade.

10.1 Âmbito e aviso

Este capítulo identifica riscos e recomendações técnicas; não constitui aconselhamento jurídico. A conformidade com o RGPD, a legislação portuguesa de proteção de dados e as obrigações fiscais exige revisão por jurista e, quando aplicável, por contabilista certificado.

10.2 Inventário de dados pessoais

Categoria	Dados	Titulares	Onde reside
Vendedor	Email, palavra-passe (hash), empresa, NIF, contactos, morada, logótipo	Representante da oficina	auth.users e empresas
Comprador	Nome, email, telefone, notas	Cliente da oficina	encomendas
Encomenda	Itens, preços, estado, previsão, histórico, fotos, modelo 3D	Comprador	encomendas, itens, histórico, fotos, Storage
Certificação	Código, critérios, garantia, número de série	Comprador/proprietário	certificados
Avaliação	Estrelas e comentário	Comprador	avaliacoes
Suporte	Mensagens e dados de diagnóstico	Utilizadores	Ainda não existe canal formal

10.3 Papéis e finalidades

No modelo white-label, a oficina é tipicamente a responsável pelo tratamento perante o comprador, e o OrbitTrack atua como subcontratante/processador. Esta qualificação depende do contrato e da realidade operacional e deve ser confirmada juridicamente. As finalidades identificáveis são: gestão da encomenda, comunicação de estado, emissão de documentos, certificação de qualidade e avaliação do serviço.

> **Base legal por confirmar**
>
> Não existe documentação que identifique a base legal para cada tratamento, o prazo de conservação ou o procedimento de resposta a titulares. A mera existência de uma página de acompanhamento não constitui base legal.

10.4 Minimização e exposição

- O portal público expõe nome do comprador, número da encomenda, estado, previsão, notas públicas, itens e fotos a quem tenha o link.
- O token é um segredo; deve ser tratado como credencial e não deve ser indexado, registado em logs ou partilhado em canais públicos.
- As RPCs devolvem IDs internos e URLs de ficheiros; devem ser minimizados.
- Os buckets públicos expõem fotos e modelos 3D a qualquer pessoa com o URL.

10.5 Direitos dos titulares

Direito	Implementação atual	Necessário
Acesso	Não existe interface de exportação.	Exportação de dados por titular.
Retificação	Perfil permite editar alguns dados; comprador não tem portal.	Procedimento e contacto.
Apagamento	Não existe.	Eliminação em cascata e anonimização.
Portabilidade	Não existe.	Exportação estruturada.
Oposição/limitação	Não existe.	Registo e procedimento.
Informação	Não existe política de privacidade.	Política pública e aviso no link.

10.6 Conservação e eliminação

Não existe prazo de conservação definido. O modelo usa ON DELETE CASCADE a partir de empresas e encomendas, mas não há rotina de eliminação, arquivo ou anonimização. Fotos e modelos 3D podem conter metadados e informações pessoais.

10.7 Subcontratantes e transferências

Os subcontratantes previsíveis são o Supabase (base de dados, autenticação e Storage) e a Vercel (alojamento da SPA). A região dos projetos, os contratos de tratamento, as transferências internacionais e as medidas de segurança devem ser confirmados e documentados. Não foi encontrada qualquer informação sobre estas escolhas no repositório.

10.8 Cookies e analítica

Não foi encontrado código de analítica, publicidade ou cookies não essenciais. Se forem adicionados, será necessário avaliar consentimento e atualizar a política.

10.9 Documentos fiscais

> **Não é faturação certificada**
>
> O OrbitTrack não emite faturas certificadas, não comunica com a AT e não integra software certificado. Recibos e certificados gerados pela oficina são documentos comerciais/qualitativos. Qualquer evolução para faturação exige validação legal, certificação de software e requisitos técnicos específicos.

10.10 Recomendações

01 Nomear responsável pela proteção de dados e mapear tratamentos (registo de atividades).
02 Definir bases legais, prazos de conservação e procedimentos de direitos.
03 Publicar política de privacidade, termos de utilização e aviso no portal.
04 Formalizar contratos com subcontratantes e verificar regiões/transferências.
05 Minimizar dados devolvidos pelas RPCs e proteger tokens em logs.
06 Implementar exportação e eliminação de dados.
07 Criar plano de resposta a incidentes e notificação.
08 Rever fiscalmente qualquer documento que possa ser interpretado como fatura.

11. Instalação e ambiente de desenvolvimento

Passos reproduzíveis, comandos reais e diagnóstico dos bloqueios atuais.

11.1 Pré-requisitos

Requisito	Versão observada / recomendada	Nota
Node.js	24.13.0 observado	Usar uma versão LTS compatível com Vite 8.
npm	11.21.1 observado	O projeto usa package-lock? Não foi confirmado; gerar e versionar.
Sistema	Windows 10/11 no ambiente auditado	Os comandos são compatíveis com macOS/Linux.
Supabase	Projeto cloud ou local	Necessário para Auth, Postgres e Storage.
Editor	VS Code ou equivalente	Extensões recomendadas: ESLint/oxlint, Tailwind, TypeScript.

11.2 Instalação de dependências

```
cd orbittrack
npm install
```

O projeto não tem repositório Git inicializado no ambiente auditado. Antes de qualquer trabalho, deve ser criado um repositório e protegido o acesso. Não existe ficheiro package-lock.json confirmado; a ausência de lockfile torna as instalações não determinísticas.

11.3 Variáveis de ambiente

Variável	Obrigatória	Exemplo seguro	Onde é usada
VITE_SUPABASE_URL	Sim	https://SEU-PROJETO.supabase.co	src/lib/supabase.ts
VITE_SUPABASE_ANON_KEY	Sim	chave anon pública	src/lib/supabase.ts
VITE_APP_URL	Não confirmada	http://localhost:5173	.env.example; não referenciada no código

Criar .env.local a partir de .env.example e preencher com valores do projeto Supabase. Nunca colocar a service_role no cliente nem versionar .env.local. O cliente lança uma exceção explícita se as duas variáveis obrigatórias faltarem.

```
# .env.local (exemplo; não versionar)
VITE_SUPABASE_URL=https://SEU-PROJETO.supabase.co
VITE_SUPABASE_ANON_KEY=SUA_CHAVE_ANON
VITE_APP_URL=http://localhost:5173
```

11.4 Aplicar o schema

01 Abrir o projeto no Supabase e o SQL Editor.

02 Confirmar que a extensão pgcrypto está disponível (gen_random_bytes). O schema só declara uuid-ossp.

03 Executar supabase/schema.sql num projeto vazio.
04 Verificar a criação das nove tabelas, das funções e dos buckets.
05 Configurar as redirect URLs de autenticação, incluindo /login e a futura rota de redefinição.
06 Executar o seed apenas depois de corrigido; o seed atual falha.

> **Idempotência**
> O schema não é idempotente. Uma segunda execução falha porque as tabelas e políticas já existem. Para evoluir o esquema, criar migrações versionadas em vez de reexecutar o ficheiro completo.

11.5 Executar em desenvolvimento

```
npm run dev
# Vite indica a URL local, por exemplo http://localhost:5173
```

O servidor de desenvolvimento pode arrancar, mas o CSS pode falhar ao compilar a classe personalizada .btn usada em @apply. O build de produção falha de forma reproduzível. Não foi possível validar a aplicação em execução contra um Supabase real.

11.6 Lint, tipos e build

Comando	Resultado observado	Conclusão
npm run lint	Código 0, 40 avisos	Não bloqueia, mas há imports mortos e avisos de hooks.
npm run build	Falha TS5101 (baseUrl)	Não gera dist.
npx vite build	Falha: Cannot apply unknown utility class btn	CSS Tailwind incompatível.
npx tsc -p tsconfig.json --noEmit --ignoreDeprecations 6.0	3 erros (import.meta.env, CSS, Cube)	Tipos incompletos.
npm audit	1 vulnerabilidade alta (image-size via pptxgenjs)	Ferramenta de documentação; não runtime do cliente.

11.7 Correzões mínimas para desbloquear o build

- Remover baseUrl do tsconfig e configurar paths de forma compatível, ou fixar uma versão de TypeScript que não trate a opção como erro.
- Adicionar "types": ["vite/client"] e uma declaração para importação de CSS, ou usar a configuração tsconfig.app correta.
- Substituir @apply btn por CSS explícito ou declarar btn como @utility no Tailwind 4.
- Remover a importação inexistente Cube de lucide-react.
- Garantir que os dois AuthProvider não são aninhados sem necessidade.

11.8 Build de produção e pré-visualização

```
npm run build
npm run preview
```

Estes comandos só serão úteis depois de corrigidos os bloqueios acima. A pré-visualização atual não pode ser gerada. Para a Vercel, será necessário um vercel.json com rewrite de SPA.

11.9 Ambiente de testes separado

- Criar um projeto Supabase dedicado a testes, com dados fictícios.
- Usar variáveis de ambiente distintas e nunca apontar testes para produção.
- Aplicar migrações e seed no projeto de testes antes de cada suite.
- Criar dois utilizadores de teste em empresas distintas para validar RLS.
- Automatizar a limpeza de dados e não usar dados pessoais reais.

11.10 Diagnóstico de erros frequentes

Sintoma	Causa provável	Resolução
“Variáveis de ambiente … são obrigatórias”	Falta .env.local ou nomes errados.	Criar o ficheiro e reiniciar o Vite.
Página pública “Não conseguimos encontrar”	RLS bloqueia consulta anon.	Ligar à RPC; não desativar RLS.
“Erro ao guardar encomenda”	numero_encomenda NOT NULL.	Gerar número no servidor.
Link de recuperação 404	Rota inexistente.	Criar /painel/redefinir-senha.
CSS não compila	@apply btn.	Corrigir Tailwind.
Tipos de import.meta.env	Falta vite/client.	Adicionar types.
Upload falha	Bucket/política ou tamanho.	Verificar Storage e limites.
Seed falha	FK zero e numero em falta.	Corrigir seed.

12. Testes e garantia de qualidade

Estado atual, plano de testes priorizado e critérios de aceitação.

12.1 Estado atual

Não existe framework de testes, ficheiros de teste, script npm test, pipeline de CI ou relatório de cobertura. O lint produz avisos, mas não bloqueia. O build falha. Consequentemente, nenhuma funcionalidade pode ser classificada como "implementada e validada".

Tipo de teste	Existe?	Observação
Unitário	Não	Sem Vitest/Jest.
Componentes	Não	Sem Testing Library.
Integração	Não	Sem base de dados de teste.
E2E	Não	Sem Playwright/Cypress.
RLS/segurança	Não	Prioridade máxima.
Acessibilidade	Não	Sem axe/Lighthouse.
Regressão visual	Não	Sem snapshots.
Carga/desempenho	Não	Sem medições.

12.2 Estratégia recomendada

01 Unitários para funções puras: formatação, validação, geração de tokens e cálculos.
02 Componentes para formulários e estados de erro/loading.
03 Integração com Supabase de teste para CRUD e RPCs.
04 RLS com dois utilizadores e duas empresas, cobrindo todos os verbos.
05 E2E dos fluxos P0: registo, criação de encomenda, link público, verificação e avaliação.
06 Acessibilidade automatizada e manual nas páginas principais.
07 Regressão visual dos ecrãs críticos em desktop e mobile.

12.3 Plano de testes priorizado

ID	Área	Cenário	Resultado esperado	Prioridade
TC-001	Auth	Registo com email novo e confirmação	Conta criada e empresa associada.	P0
TC-002	Auth	Login com credenciais válidas	Sessão iniciada e /painel acessível.	P0
TC-003	Auth	Login com palavra-passe errada	Erro genérico, sem revelar existência.	P1
TC-004	Auth	Recuperação de palavra-passe completa	Nova palavra-passe aceite e login funciona.	P0
TC-005	RLS	A lê encomendas de B	Zero linhas.	P0
TC-006	RLS	A atualiza encomenda de B	Operação rejeitada.	P0
TC-007	RLS	A insere encomenda com empresa_id de B	Operação rejeitada.	P0
TC-008	Storage	A tenta escrever na pasta de B	Operação rejeitada.	P0
TC-009	Storage	Anon acede a foto de encomenda	Acesso negado ou URL assinada inválida.	P0
TC-010	Encomendas	Criar encomenda com 2 itens	Número gerado, itens persistidos.	P0

ID	Área	Cenário	Resultado esperado	Prioridade
TC-011	Encomendas	Criar duas encomendas em simultâneo	Números distintos.	P1
TC-012	Encomendas	Editar encomenda existente	Dados carregados e atualizados.	P0
TC-013	Encomendas	Atualizar estado com mensagem/foto	Histórico criado e portal reflete.	P0
TC-014	Público	Abrir link com token válido	Dados corretos, sem login.	P0
TC-015	Público	Abrir link com token inválido	Estado de erro controlado.	P0
TC-016	Público	Previsão nula	Mostra “a definir”, não 1970.	P1
TC-017	Passaporte	Abrir /peca/:tokenPeca	Ficha e modelo corretos.	P1
TC-018	3D	Modelo GLB válido	Cena carrega e permite orbitar.	P1
TC-019	3D	URL inválido	Fallback sem rebentar a página.	P1
TC-020	Certificados	Verificar código válido	Empresa, peça, data e selo.	P0
TC-021	Certificados	Código minúsculo/sem hífen	Normalização e resultado válido.	P1
TC-022	PDF	Gerar recibo com itens	PDF com marca, valores e numeração.	P1
TC-023	PDF	Gerar certificado com QR	PDF com selo, critérios e QR.	P1
TC-024	Avaliações	Submeter avaliação duas vezes	Segunda tentativa rejeitada.	P1
TC-025	Avaliações	Estrelas fora de 1–5	Erro controlado.	P1
TC-026	Upload	Ficheiro com 51 MB	Rejeitado com mensagem.	P1
TC-027	Upload	Ficheiro .exe renomeado para .png	Rejeitado por MIME/conteúdo.	P1
TC-028	Dashboard	Sem encomendas	Estado vazio, sem métricas falsas.	P1
TC-029	Mobile	Portal em 375 px	Sem overflow; timeline legível.	P1
TC-030	A11y	Navegação só com teclado	Foco visível e ordem lógica.	P2
TC-031	Build	npm run build	Código 0 e dist gerado.	P0
TC-032	Deploy	Abrir rota profunda na Vercel	SPA carrega, sem 404.	P1

12.4 Exemplo de teste RLS

```
-- Executar num projeto de testes, com dois utilizadores A e B.
-- Autenticar como A e tentar ler a encomenda de B.
select *
from encomendas
where id = '<ID_DA_ENCOMENDA_DE_B>';
-- Esperado: 0 linhas.

-- Autenticar como A e tentar atualizar a encomenda de B.
update encomendas
set estado = 'Entregue'
where id = '<ID_DA_ENCOMENDA_DE_B>';
-- Esperado: 0 linhas afetadas (ou erro de política).
```

Estes testes devem ser repetidos para todas as tabelas e para operações de INSERT/UPDATE/DELETE. A evidência (output, data, ambiente) deve ser arquivada.

12.5 Testes de componentes e E2E

- Componentes: formulário de encomenda, perfil, configuração de estados, cartões de estado e timeline.
- E2E: registo → perfil → encomenda → estado → link público → avaliação.
- E2E de segurança: tentar abrir /painel sem sessão e aceder a IDs de outra empresa.
- E2E de documentos: descarregar recibo e certificado e validar conteúdo com pdf-parse.

- E2E de acessibilidade: axe em cada página e testes de teclado.

12.6 Critérios de saída

Gate	Critério
G0 — Build	npm run build e lint verdes em CI.
G1 — Segurança	Testes RLS/Storage verdes; nenhuma falha crítica aberta.
G2 — Fluxos	Todos os P0 com E2E verde.
G3 — Documentos	PDFs validados e QR verificado.
G4 — Acessibilidade	WCAG 2.2 AA nas páginas principais.
G5 — Operação	Monitorização, backups e rollback testados.

Estado atual dos gates

G0 está bloqueado. G1 não foi executado. G2 não é possível porque os fluxos P0 não funcionam. G3, G4 e G5 não têm implementação.

13. Deploy, manutenção e operações

O que existe, o que falta e como operar o produto quando os bloqueios forem resolvidos.

13.1 Estado atual

Item	Estado	Evidência
Repositório Git	Não inicializado no ambiente auditado	Ausência de .git.
CI/CD	Não existe	Sem .github/workflows.
Build	Falha	TS5101 e @apply btn.
Vercel	Não configurada	Sem vercel.json.
Supabase	Schema manual	Sem migrações.
Monitorização	Não existe	Sem Sentry, logs ou alertas.
Backups	Não documentados	Depende do plano Supabase.
Domínio/TLS	Não configurados	Sem informação no repositório.

13.2 Ambientes

Ambiente	Frontend	Supabase	Dados	Objetivo
Local	Vite	Projeto de desenvolvimento	Fictícios	Desenvolvimento.
Teste	Preview Vercel	Projeto de teste	Fictícios e isolados	CI, RLS, E2E.
Produção	Vercel	Projeto de produção	Reais	Clientes.

Os três ambientes devem usar projetos Supabase distintos. Misturar produção e teste é um risco de perda de dados e de fuga de informação.

13.3 Configuração Vercel recomendada

```
{
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ],
  "headers": [
    {
      "source": "/(.*)",
      "headers": [
        { "key": "X-Content-Type-Options", "value": "nosniff" },
        { "key": "Referrer-Policy", "value": "strict-origin-when-cross-origin" },
        { "key": "X-Frame-Options", "value": "DENY" }
      ]
    }
  ]
}
```

Este ficheiro não existe no projeto; é uma recomendação. Uma CSP completa deve ser desenhada depois de inventariar os domínios do Supabase e eventuais fontes externas.

13.4 Variáveis por ambiente

Variável	Local	Teste	Produção
VITE_SUPABASE_URL	Projeto dev	Projeto teste	Projeto prod
VITE_SUPABASE_ANON_KEY	Chave dev	Chave teste	Chave prod
VITE_APP_URL	http://localhost:5173	URL de preview	URL pública

As variáveis VITE_* são incorporadas no bundle e não são segredos. A service_role nunca deve ser exposta. Chaves devem ser rotáveis e o acesso ao dashboard Supabase restringido.

13.5 Processo de release

01 Criar branch a partir de main e abrir pull request.
02 Executar lint, tipos, testes unitários, RLS e build em CI.
03 Aplicar migrações no ambiente de teste e correr E2E.
04 Gerar preview deployment e validar smoke tests.
05 Aprovar revisão de código e de segurança.
06 Aplicar migrações em produção com plano de rollback.
07 Publicar frontend e monitorizar erros e métricas.
08 Registar a versão e as alterações.

13.6 Migrações

- Substituir schema.sql por migrações numeradas e imutáveis.
- Nunca alterar uma migração já aplicada; criar uma nova.
- Testar em cópia de produção antes de aplicar.
- Fazer backup antes de alterações destrutivas.
- Manter scripts de rollback para cada migração.
- Documentar dependências e ordem de execução.

13.7 Monitorização e logs

Sinal	Ferramenta sugerida	Alerta
Erros de frontend	Sentry ou equivalente	Novo erro ou aumento de taxa.
Erros de API/PostgREST	Logs Supabase	5xx, 401/403 anómalos.
Latência	Vercel Analytics / Supabase	P95 acima do objetivo.
Falhas de build	CI	Bloquear merge.
Abuso de RPC	Rate limit + logs	Picos por IP/token.
Armazenamento	Supabase	Crescimento anómalo.
Backups	Supabase	Falha ou atraso.

13.8 Backups e recuperação

O plano do Supabase contratado determina a retenção e a frequência dos backups. Não foi encontrada documentação sobre RPO/RTO. Antes de produção, definir objetivos, testar restauro e documentar o procedimento. Dados de Storage também precisam de estratégia de backup.

13.9 Rollback

01 Reverter o deployment do frontend para a versão anterior na Vercel.
02 Reverter a migração com o script preparado, se seguro.
03 Se os dados estiverem corrompidos, restaurar backup e comunicar o incidente.
04 Invalidar sessões/chaves se houver suspeita de compromisso.
05 Registar causa, impacto e ações corretivas.

13.10 Manutenção

- Atualizar dependências mensalmente e rever vulnerabilidades.
- Revisar políticas RLS e Storage a cada alteração de modelo.
- Rodar testes de restauro trimestralmente.
- Rever custos e quotas.
- Manter documentação e runbooks atualizados.
- Planear janelas de manutenção para migrações destrutivas.

> **Não existe operação pronta**
>
> Sem build, CI, monitorização, backups documentados ou plano de rollback, o OrbitTrack não tem uma operação de produção. Este capítulo descreve o destino necessário, não o estado atual.

14. Desempenho e escalabilidade

Pontos de pressão, medições disponíveis e recomendações, distinguindo factos de estimativas.

14.1 O que foi medido

Não foi possível medir desempenho em execução: o build falha e não existe ambiente Supabase configurado. As observações abaixo são estáticas, baseadas no código e no esquema. Não são apresentados valores de latência, throughput ou custos inventados.

14.2 Pontos de pressão

Área	Observação	Impacto potencial
Bundle inicial	Todas as páginas e three.js são importados estaticamente em App.tsx.	Carregamento inicial pesado, sobretudo em mobile.
Lista de encomendas	SELECT sem paginação nem filtros.	Resposta lenta com volume.
RPC pública	Agrega itens, histórico, fotos, certificados e avaliação num único JSON.	Resposta grande e custo de agregação.
Índices	Bom conjunto para tokens e FKs; falta índice de ordenação por criada_em.	Ordenações podem ficar lentas.
Storage	Ficheiros 3D sem limite e servidos publicamente.	Consumo de banda e custos.
PDFs	Gerados no cliente, sem limite de complexidade.	Bloqueio do browser em documentos grandes.
3D	Modelos sem limite de tamanho nem compressão.	Falhas de memória em telemóveis.
Dashboard	Count exact em todos os pedidos.	Custo crescente com volume.

14.3 Consultas e índices

- encomendas tem índices por empresa e token; a listagem ordenada por criada_em beneficiaria de índice composto (empresa_id, criada_em DESC).
- itens, histórico, fotos e certificados têm índices por encomenda/item.
- A RPC pública faz várias subqueries correlacionadas; com muitas fotos/itens pode ser pesada.
- Não há pesquisa textual; se for adicionada, avaliar pg_trgm ou full-text.
- O count exact do dashboard pode ser substituído por contagem estimada ou cache.

14.4 Paginação e carregamento

Nenhuma listagem é paginada. A primeira otimização deve ser paginação server-side com limites máximos, seguida de seleção explícita de colunas. O portal público deve limitar o número de fotos e itens devolvidos ou paginar secções.

14.5 Frontend

- Introduzir lazy loading por rota, especialmente para o visualizador 3D e páginas públicas.
- Separar o bundle de three.js e carregá-lo apenas quando necessário.
- Evitar re-renderizações desnecessárias e pedidos duplicados.
- Adicionar cache de dados (React Query ou equivalente) para listas e perfil.

- Otimizar imagens (largura, formato WebP, lazy loading).
- Medir Core Web Vitals depois de o build funcionar.

14.6 Escalabilidade multi-empresa

O modelo é single-tenant por utilizador: cada empresa tem um dono e não há membros. Isto simplifica o isolamento, mas limita equipas. O volume por empresa é desconhecido. As políticas RLS com subqueries são adequadas para volumes moderados; devem ser testadas com dados representativos.

14.7 Custos

Os custos dependem dos planos Supabase e Vercel, do armazenamento de modelos 3D, do tráfego e do número de pedidos. Não são apresentados valores porque não foram medidos nem contratados. Antes de comercializar, modelar cenários de uso e definir alertas de faturação.

14.8 Objetivos sugeridos

Métrica	Objetivo inicial (a validar)
LCP mobile	≤ 2,5 s em 4G
Resposta da lista	≤ 500 ms para 50 registos
Resposta do portal	≤ 800 ms com 20 fotos
Erros 5xx	< 0,5% dos pedidos
Tamanho do modelo 3D	≤ 15 MB comprimido
Upload de imagem	≤ 5 MB após compressão

> **Estimativas, não garantias**
> Os objetivos acima são propostas de engenharia. Não foram medidos nem validados com utilizadores reais.

15. Limitações, riscos e dívida técnica

Inventário priorizado, com evidência, impacto, recomendação e critério de conclusão.

15.1 Método

Os riscos abaixo foram confirmados por leitura de código ou por execução de comandos. A coluna "Probabilidade" distingue o que já acontece (Confirmada) do que é provável mas não foi reproduzido. O esforço é indicado em S/M/L, sem datas, porque não existe equipa dimensionada nem histórico de velocidade.

15.2 Resumo priorizado

ID	Problema	Gravidade	Prob.	Prioridade	Esforço
R-01	Build de produção bloqueado	Crítica	Confirmada	P0	M
R-02	Páginas públicas bloqueadas pela RLS	Crítica	Confirmada	P0	M
R-03	Criação de encomenda falha por numero_encomenda NOT NULL	Crítica	Confirmada	P0	M
R-04	Buckets públicos expõem ficheiros de encomendas	Crítica	Confirmada	P0	M
R-05	Políticas de Storage demasiado permissivas	Crítica	Confirmada	P0	M
R-06	Recuperação de palavra-passe sem rota de destino	Alta	Confirmada	P1	S
R-07	Sem testes de isolamento RLS	Crítica	Alta	P0	M
R-08	RPCs SECURITY DEFINER sem search_path fixo	Alta	Média	P1	S
R-09	Numeração concorrente pode duplicar encomendas/recibos	Alta	Média	P1	M
R-10	Código de certificado gerado com RANDOM()	Média	Média	P2	S
R-11	Seed inválido	Alta	Confirmada	P1	S
R-12	Dashboard apresenta métricas fixas	Alta	Confirmada	P1	S
R-13	Limites de upload não aplicados	Alta	Alta	P1	S
R-14	Sem rate limiting nas RPCs públicas	Média	Alta	P2	M
R-15	Sem testes, CI ou verificação de tipos	Alta	Confirmada	P1	M
R-16	Sem rewrites SPA na Vercel	Média	Confirmada	P1	S
R-17	Dados pessoais sem base documental de conformidade	Alta	Alta	P1	L
R-18	Sem logs de auditoria de negócio	Média	Alta	P2	M
R-19	Listagens e RPCs sem paginação	Média	Média	P2	M
R-20	Acessibilidade não validada	Média	Alta	P2	M

15.3 Riscos críticos em detalhe

R-01 — Build de produção bloqueado

Campo	Detalhe
Evidência	npm run build: TS5101 (baseUrl); npx vite build: "Cannot apply unknown utility class btn"; tsc: import.meta.env, index.css e Cube
Impacto	Não é possível publicar na Vercel nem garantir tipos; o produto não tem artefacto de produção.
Gravidade / probabilidade	Crítica / Confirmada

Campo	Detalhe
Recomendação	Corrigir tsconfig/types, declarar CSS e substituir o @apply de classes personalizadas por @utility ou CSS explícito.
Dependência	Nenhuma
Critério de conclusão	npm run build termina com código 0 e gera dist/.

R-02 — Páginas públicas bloqueadas pela RLS

Campo	Detalhe
Evidência	AcompanharEncomenda.tsx e VerificarCertificado.tsx consultam tabelas como anon; não existem políticas anon
Impacto	O comprador não consegue acompanhar encomendas nem verificar certificados.
Gravidade / probabilidade	Crítica / Confirmada
Recomendação	Ligar as páginas às RPCs obter_encomenda_publica, obter_peca_publica e verificar_certificado.
Dependência	RPCs testadas
Critério de conclusão	Link público abre com dados reais e token inválido mostra erro controlado.

R-03 — Criação de encomenda falha por numero_encomenda NOT NULL

Campo	Detalhe
Evidência	EncomendaForm.tsx não envia numero_encomenda; schema.sql exige NOT NULL; gerar_numero_encomenda não é usada
Impacto	O fluxo central do vendedor não funciona.
Gravidade / probabilidade	Crítica / Confirmada
Recomendação	Gerar número de forma transacional no servidor e inserir encomenda + itens numa única operação.
Dependência	R-01
Critério de conclusão	Criar encomenda com 2 itens e ver o número 2026-0001 na lista.

R-04 — Buckets públicos expõem ficheiros de encomendas

Campo	Detalhe
Evidência	schema.sql: storage.buckets public=true para empresas e encomendas
Impacto	Fotos, modelos 3D e documentos podem ser acedidos por qualquer pessoa com o URL.
Gravidade / probabilidade	Crítica / Confirmada
Recomendação	Bucket encomendas privado, URLs assinadas e validação de propriedade no Storage.
Dependência	Nenhuma
Critério de conclusão	Acesso anónimo a um ficheiro de encomenda devolve 400/403; URLs assinadas expiram.

R-05 — Políticas de Storage demasiado permissivas

Campo	Detalhe
Evidência	Qualquer utilizador autenticado pode INSERT/UPDATE/DELETE em qualquer caminho dos buckets
Impacto	Um vendedor pode sobrescrever ou apagar ficheiros de outra empresa.
Gravidade / probabilidade	Crítica / Confirmada

Campo	Detalhe
Recomendação	Restringir caminhos por empresa_id/auth.uid() e separar operações.
Dependência	R-04
Critério de conclusão	Teste com dois utilizadores prova que o segundo não consegue escrever na pasta do primeiro.

R-07 — Sem testes de isolamento RLS

Campo	Detalhe
Evidência	Não existem testes; políticas não foram executadas contra a base
Impacto	Não é possível afirmar que os dados estão isolados entre empresas.
Gravidade / probabilidade	Crítica / Alta
Recomendação	Criar testes com dois utilizadores e tentar SELECT/UPDATE/DELETE cruzados.
Dependência	Projeto Supabase de testes
Critério de conclusão	Suite RLS verde e evidência arquivada.

15.4 Riscos de prioridade 1 e 2

ID	Evidência	Impacto	Recomendação	Critério
R-06	resetPassword redireciona para /painel/redefinir-senha; App.tsx não tem essa rota	Utilizador que perde a palavra-passe fica bloqueado.	Criar página de redefinição, tratar o token e testar o fluxo completo.	Pedido de recuperação permite definir nova palavra-passe e iniciar sessão.
R-08	Funções em schema.sql sem SET search_path	Risco de resolução maliciosa de objetos; alerta habitual do linter Supabase.	Adicionar SET search_path = public, pg_temp e revisar permissões.	Linter sem alertas e testes funcionais verdes.
R-09	gerar_numero_encomenda/gerar_numero_recibo usam MAX+1; recibo sem UNIQUE	Erros em picos de utilização e documentos duplicados.	Sequências por empresa/ano ou transação com lock; adicionar constraints.	Teste concorrente de 50 criações sem duplicados.
R-10	gerar_codigo_verificacao usa RANDOM()	Menor entropia e previsibilidade; colisões não tratadas.	Usar gen_random_bytes e retry; manter formato legível.	Códigos gerados com CSPRNG e teste de colisão.
R-11	seed.sql: utilizador_id zero e numero_encomenda em falta	Ambiente de demonstração não pode ser criado como documentado.	Criar seed idempotente com utilizador de teste e dados completos.	Seed executa duas vezes sem erro e gera demo navegável.
R-12	Dashboard.tsx: tempoMedio "3 dias", avaliacaoMedia 4.8	Decisões comerciais com dados falsos; quebra de confiança.	Remover valores ou calcular a partir de dados reais com estado vazio.	Métricas calculadas ou substituídas por "sem dados".
R-13	utils.ts tem validadores e limites, mas EmpresaPerfil não os usa	Ficheiros enormes ou maliciosos; custos e risco de abuso.	Validar MIME/extensão/tamanho no cliente e no Storage.	Upload de .exe e de ficheiro > limite é rejeitado com mensagem clara.
R-14	GRANT EXECUTE TO anon sem limitação	Abuso, scraping e custos; verificação de certificados enumerável.	Aplicar rate limit no edge/função e monitorizar.	Limite por IP/token documentado e testado.
R-15	package.json sem script test; sem .github/workflows	Regressões entram sem deteção; o build já está quebrado.	Adicionar Vitest, Playwright, lint e build em CI.	Pipeline bloqueia merge com build/testes falhados.
R-16	Não existe vercel.json	Links profundos devolvem 404 em produção.	Adicionar rewrite para /index.html e testar rotas públicas.	Abrir /acompanhar/token diretamente devolve a SPA.
R-17	Sem política de privacidade, termos, retenção ou direitos	Risco RGPD e de confiança comercial.	Mapear tratamentos, contratos, políticas e procedimentos.	Documentação revista por jurista e publicada.
R-18	Apenas histórico de estados; sem registo de acessos/ações	Difícil investigar incidentes e alterações.	Registar eventos críticos com ator, ação, entidade e timestamp.	Eventos críticos consultáveis e protegidos.

ID	Evidência	Impacto	Recomendação	Critério
R-19	EncomendasLista seleciona tudo; RPC agrega todos os itens/fotos	Degradação com volume e respostas grandes.	Paginação server-side e limites nas RPCs.	Lista e portal respondem em tempo aceitável com 1.000 encomendas.
R-20	Botões de ícone sem aria-label; tabela sem scope; lang="en" no index.html	Barreiras para leitores de ecrã e incumprimento de boas práticas.	Auditoria axe/Lighthouse, correções e testes com teclado.	WCAG 2.2 AA verificada nas páginas principais.

15.5 Dívida técnica por categoria

Categoria	Exemplos	Consequência
Build e tipos	baseUrl, vite/client, CSS, Cube	Não há artefacto de produção.
Dados	Numeração, constraints, seed	Fluxos falham e há risco de duplicados.
Segurança	Buckets públicos, Storage, RPC	Isolamento não comprovado.
Frontend	Estado local, duplicação, imports mortos	Manutenção difícil e bugs silenciosos.
Testes	Ausência total	Regressões inevitáveis.
Operações	CI, monitorização, backups	Não há produção sustentável.
Conformidade	RGPD, termos, retenção	Risco legal e comercial.

15.6 O que não pode ser afirmado

> **Afirmações proibidas no estado atual**
>
> Não é possível afirmar que o OrbitTrack está pronto para produção, que o isolamento multi-empresa está validado, que os dados estão seguros, que o portal do comprador funciona, que os certificados são verificáveis, que os PDFs são emitidos, que há conformidade RGPD ou que o produto é “Vercel-ready”.

15.7 Ordem de correção recomendada

01 Desbloquear o build e a verificação de tipos.
02 Corrigir a criação de encomendas com numeração transacional.
03 Ligar o portal público e a verificação às RPCs.
04 Fechar o Storage: buckets privados, políticas por dono e validação de uploads.
05 Criar testes RLS/Storage com dois utilizadores.
06 Implementar atualização de estado, histórico e fotos.
07 Implementar recibos, certificados, QR e passaporte.
08 Adicionar CI, monitorização, backups e documentação legal.

16. Roadmap do produto

Fases de correção, segurança, conclusão funcional, qualidade e preparação comercial, sem datas inventadas.

16.1 Princípios

- Nenhuma fase avança enquanto os critérios de saída da anterior não estiverem cumpridos.
- Segurança e integridade de dados precedem novas funcionalidades.
- Cada funcionalidade comercialmente prometida tem de ter testes e evidência.
- Não são atribuídas datas nem estimativas de semanas sem equipa e histórico.
- O esforço é classificado em S (pequeno), M (médio) e L (grande).

16.2 Fase 1 — Estabilidade e build

Item	Ação	Critério de saída	Esforço
Build	Corrigir tsconfig, tipos e Tailwind.	npm run build = 0.	M
Repositório	Inicializar Git, proteger main e criar PR template.	Histórico versionado.	S
Encomendas	Gerar numero_encomenda no servidor; transação com itens.	Criar encomenda com itens.	M
Seed	Corrigir FK e campos obrigatórios; tornar idempotente.	Seed executável duas vezes.	S
Recuperação	Criar rota e formulário de nova palavra-passe.	Fluxo completo validado.	S

16.3 Fase 2 — Segurança e integridade

Item	Ação	Critério de saída	Esforço
Storage	Buckets privados, políticas por empresa e URLs assinadas.	Teste cruzado negado.	M
Uploads	Validar MIME, extensão e tamanho.	Ficheiros inválidos rejeitados.	S
RPCs	SET search_path, validação e limites.	Linter e testes verdes.	S
RLS	Suite com dois utilizadores em todas as tabelas.	Evidência arquivada.	M
Numeração	Sequências/transações e constraints.	Teste concorrente sem duplicados.	M
Rate limit	Limitar RPCs públicas e Auth.	Abuso controlado.	M

16.4 Fase 3 — Fluxos essenciais

Item	Ação	Critério de saída	Esforço
Portal público	Ligar à RPC e tratar estados.	Link válido e inválido funcionam.	M
Verificação	Ligar à RPC e usar :codigo.	Certificado apresenta todos os campos.	S
Estado + fotos	Atualização transacional e timeline.	Portal reflete alteração.	M
Passaporte	Corrigir rota e consumir RPC.	Página completa com ficha.	M
3D	STL, error boundary e fallback.	GLB/STL e erro controlado.	M
PDFs	Recibo e certificado com marca e QR.	Documentos validados.	L

Item	Ação	Critério de saída	Esforço
QR	PNG/PDF e cartão.	Download e leitura corretos.	M
Avaliações	Formulário e regras.	Avaliação única e visível.	S

16.5 Fase 4 — Qualidade

Item	Ação	Critério de saída	Esforço
Unitários	Funções puras e formatadores.	Cobertura definida.	S
Componentes	Formulários e estados.	Testes verdes.	M
E2E	Fluxos P0 com Playwright.	Pipeline estável.	M
A11y	axe + testes manuais.	WCAG 2.2 AA.	M
Performance	Lazy loading e paginação.	Orçamentos cumpridos.	M

16.6 Fase 5 — Produção

Item	Ação	Critério de saída	Esforço
CI/CD	Pipeline com lint, tipos, testes e build.	Merge bloqueado se falhar.	M
Vercel	Rewrites, headers e preview.	Rotas profundas funcionam.	S
Monitorização	Erros, latência e alertas.	Dashboard operacional.	M
Backups	Política, teste de restauro.	RPO/RTO documentados.	M
Runbooks	Incidentes, rollback e manutenção.	Equipa treinada.	S

16.7 Fase 6 — Validação comercial

Item	Ação	Critério de saída	Esforço
Piloto	2–3 oficinas com dados reais e consentimento.	Feedback estruturado.	M
Preços	Definir fatores e modelo; não inventar valores.	Proposta aprovada.	M
Suporte	Canal, SLA e base de conhecimento.	Processo documentado.	M
Marca	Alinhar aplicação com identidade OrbitTrack.	Tema e logótipo consistentes.	S
Legal	Política, termos, DPA e retenção.	Revisão jurídica.	L

16.8 Fase 7 — Evolução

Item	Ação	Pré-requisito	Esforço
Notificações	Email/WhatsApp com filas e templates.	Produção estável.	L
Pagamentos	Stripe/MB Way e reconciliação.	Modelo de preços.	L
Equipas	Membros, papéis e convites.	Modelo RLS revisto.	L

Item	Ação	Pré-requisito	Esforço
Integração com impressoras	OctoPrint/PrusaConnect ou similar.	Casos de uso validados.	L
Faturação	Software certificado e requisitos legais.	Parceria/validação.	L
Analítica	Métricas de produto e relatórios.	Eventos definidos.	M

16.9 Dependências entre fases

Fase	Depende de	Porquê
1	—	Sem build não há entrega.
2	1	Testar segurança exige app executável.
3	1, 2	Fluxos precisam de dados e isolamento.
4	3	Só se testa o que existe.
5	1–4	Produção exige qualidade.
6	5	Piloto exige ambiente estável.
7	6	Evolução exige validação comercial.

> **Sem datas**
>
> O roadmap não inclui datas. Qualquer compromisso comercial deve ser assumido apenas depois de a equipa medir a sua velocidade real e os bloqueios externos.

17. Operação e suporte

Perguntas frequentes, diagnóstico, mensagens conhecidas e procedimentos de recuperação.

17.1 Âmbito do suporte

Não existe canal de suporte formal, SLA, base de conhecimento ou equipa de operações. Este capítulo propõe um modelo mínimo para quando o produto entrar em piloto.

Nível	Responsabilidade	Exemplo
N1 — Autosserviço	Documentação, FAQ e estados de erro claros.	Não consigo iniciar sessão.
N2 — Suporte funcional	Triagem, recolha de informação e resolução de configuração.	Link público não abre.
N3 — Engenharia	Bugs, dados, segurança e infraestrutura.	RLS ou migração falhada.
N4 — Jurídico/segurança	Incidentes de dados e obrigações legais.	Fuga de informação.

17.2 Perguntas frequentes

Pergunta	Resposta
O comprador precisa de conta?	Não. O acesso é feito por link com token; não há registo de comprador.
Onde encontro o link da encomenda?	O token existe na base de dados, mas a UI para copiar/partilhar ainda não está implementada.
Posso mudar as cores da minha empresa?	Sim, no perfil. A aplicação ainda não aplica essas cores em todos os documentos.
Posso alterar o estado de uma encomenda?	Ainda não existe ecrã funcional para o fazer.
O certificado é uma certificação legal?	Não. É um selo de qualidade da oficina, verificado por código.
O recibo serve para efeitos fiscais?	Não. Não é uma fatura certificada.
O modelo 3D é público?	O bucket é público no estado atual; deve ser tornado privado antes de produção.
Posso apagar a minha conta?	Não existe fluxo de eliminação.
O sistema guarda o histórico?	O modelo prevê histórico, mas a aplicação ainda não o escreve.
Que formatos 3D são suportados?	GLB e GLTF no componente atual; STL está previsto.
Como recupero a palavra-passe?	O pedido é enviado, mas o link abre uma rota inexistente; é um bloqueio conhecido.
O produto está pronto para vender?	Não. O build falha, os fluxos públicos estão bloqueados e não há testes.

17.3 Problemas comuns

Sintoma	Diagnóstico	Resolução
Página em branco no painel	Erro de env ou sessão.	Ver consola, .env.local e sessão.
“Não conseguimos encontrar esta encomenda”	RLS a bloquear consulta anon.	Ligar à RPC; não desativar RLS.

Sintoma	Diagnóstico	Resolução
Link de recuperação 404	Rota em falta.	Implementar /painel/redefinir-senha.
Erro ao guardar encomenda	numero_encomenda em falta.	Implementar numeração transacional.
Dashboard com 3 dias e 4.8	Valores fixos.	Substituir por dados reais ou “sem dados”.
Upload falha	Política/tamanho.	Verificar bucket, limites e MIME.
Tabela transparente/estilos ausentes	CSS Tailwind falhou.	Corrigir @apply btn.
Build falha	TS5101 + CSS.	Seguir Capítulo 11.

17.4 Mensagens de erro conhecidas

Mensagem	Origem	Significado
“Email ou palavra-passe incorretos.”	Login.tsx	Credenciais inválidas; não distingue email de palavra-passe.
“Erro ao guardar encomenda.”	EncomendaForm.tsx	Falha genérica; inclui numero_encomenda em falta.
“Não conseguimos encontrar esta encomenda. Verifica o link.”	AcompanharEncomenda.tsx	Token inválido ou consulta bloqueada por RLS.
“Código Inválido ou não encontrado”	VerificarCertificado.tsx	Consulta falhou ou certificado inexistente.
“Variáveis de ambiente … são obrigatórias.”	supabase.ts	Falta configuração local.
“Certificado Válido”	VerificarCertificado.tsx	Consulta direta devolveu registo (quando autenticado).

17.5 Procedimentos de recuperação

Cenário	Procedimento
Palavra-passe perdida	Corrigir rota de redefinição; até lá, criar nova conta ou redefinir via Supabase Auth.
Empresa não criada após registo	Verificar trigger e criar registo manualmente com utilizador_id correto.
Dados apagados por engano	Restaurar backup; não editar diretamente em produção sem auditoria.
Chave comprometida	Rotar anon/service_role, invalidar sessões e rever logs.
Ficheiro público exposto	Tornar bucket privado, invalidar URLs e avaliar notificação.
Build quebrado	Reverter para última versão funcional; corrigir em branch.

17.6 Como reportar um problema

Recolher, sem incluir segredos: URL, data/hora, utilizador (email), empresa, navegador, passos, resultado esperado, resultado obtido, captura de ecrã e mensagens de consola. Nunca enviar palavras-passe, tokens, chaves ou dados pessoais desnecessários.

17.7 Critérios de escalonamento

- Suspeita de acesso não autorizado ou fuga de dados: escalar imediatamente para segurança/jurídico.
- Perda de dados ou corrupção: N3 imediato e ativar backups.
- Build ou produção em baixo: N3 e rollback.
- Bloqueio de um cliente sem alternativa: N2/N3 conforme impacto.

- Pedido de eliminação/exportação: jurídico e privacidade.

17.8 Limites do suporte

Não há SLA, horário alargado, telefone ou gestão de incidentes 24/7. O suporte não pode prometer funcionalidades planeadas nem contornar a RLS. Alterações de dados em produção devem ser excecionais, auditadas e aprovadas.

18. Glossário e anexos

Termos, matrizes, referências, dependências, histórico e pontos por validar.

18.1 Glossário técnico e funcional

Termo	Definição
Auth	Serviço de autenticação do Supabase.
BaaS	Backend as a Service; serviços geridos em vez de servidor próprio.
Bucket	Contentor de ficheiros no Storage.
Constraint	Regra de integridade no PostgreSQL (UNIQUE, CHECK, FK).
CSP	Content Security Policy; cabeçalho que limita origens de scripts.
E2E	Teste ponta a ponta de um fluxo completo.
Edge Function	Função serverless no edge; não usada no projeto.
JWT	Token assinado que identifica a sessão.
MFA	Autenticação multifator.
Migração	Alteração versionada ao esquema da base de dados.
PostgREST	Camada que expõe tabelas/funções PostgreSQL via HTTP.
RGPD	Regulamento Geral sobre a Proteção de Dados.
RLS	Row Level Security do PostgreSQL.
RPC	Remote Procedure Call; função PostgreSQL exposta.
SaaS	Software as a Service.
Service role	Chave Supabase com privilégios totais; nunca no cliente.
SLA	Acordo de nível de serviço.
SPA	Single Page Application.
Tenant	Empresa isolada logicamente.
Token	Cadeia aleatória usada como credencial de acesso.
White-label	Produto com a marca do vendedor.

18.2 Abreviaturas

Abreviatura	Significado
AT	Autoridade Tributária e Aduaneira.
CNPD	Comissão Nacional de Proteção de Dados.
DPA	Data Processing Agreement.
FK	Foreign Key.
PK	Primary Key.
RGPD	Regulamento (UE) 2016/679.
RPO/RTO	Recovery Point Objective / Recovery Time Objective.

Abreviatura	Significado
WCAG	Web Content Accessibility Guidelines.

Anexo A — Matriz de funcionalidades

Esta matriz é a fonte de verdade para a apresentação e para a documentação. Os estados seguem as seis categorias definidas no início do projeto.

Módulo	Funcionalidade	Estado	Evidência	Limitações	Próxima ação
Autenticação	Início de sessão com email e palavra-passe	Implementada, mas não validada	src/pages/Login.tsx; src/contexts/AuthContext.tsx (signIn); Supabase Auth	Não existem testes nem execução contra um projeto Supabase real. A mensagem de erro é genérica.	Criar projeto de testes, validar confirmação de email, implementar testes E2E e melhorar mensagens.
Autenticação	Registo de vendedor e criação automática da empresa	Implementada, mas não validada	src/pages/Registar.tsx; trigger criar_empresa_para_utilizador em supabase/schema.sql	O trigger em auth.users é frágil e não foi validado; empresas.utilizador_id não tem restrição UNIQUE.	Testar em projeto limpo, tornar o trigger idempotente e adicionar UNIQUE(utilizador_id).
Autenticação	Recuperação de palavra-passe	Bloqueada ou com problemas	src/contexts/AuthContext.tsx (resetPassword) redireciona para /painel/redefinir-senha; src/App.tsx não define essa rota	O link de recuperação abre uma rota inexistente. Não existe ecrã para definir a nova palavra-passe.	Criar rota e formulário de redefinição, tratar o evento PASSWORD_RECOVERY e testar o fluxo completo.
Autenticação	Proteção de rotas do painel	Implementada, mas não validada	src/components/RotaProtegida.tsx; src/App.tsx	Proteção apenas no cliente; não há testes de sessão expirada nem de acesso direto a rotas.	Testar sessão ausente/expirada e garantir que a RLS é a barreira efetiva no servidor.
Empresa	Perfil white-label: nome, descrição, contactos, cores e rodapé	Implementada, mas não validada	src/pages/painel/EmpresaPerfil.tsx; tabela empresas em supabase/schema.sql	Sem testes; validação apenas HTML; não há pré-visualização real da página pública.	Validar gravação/leitura, validar cores e contactos e adicionar testes de componente.
Empresa	Upload de logótipo e foto de capa	Parcialmente implementada	EmpresaPerfil.tsx (handleFileUpload); bucket empresas	Aceita qualquer tipo/tamanho; bucket público; políticas de Storage não restringem a pasta da empresa.	Validar MIME e tamanho, tornar o bucket privado ou restringir caminhos e aplicar URLs assinadas.
Estados	Configuração de estados por empresa (nome, cor, ordem, mensagem)	Implementada, mas não validada	src/pages/painel/EstadosConfig.tsx; tabela configuracao_estados; trigger criar_estados_padrao	Sem testes; não há unicidade de nome/ordem; a reordenação só é persistida ao guardar.	Adicionar restrições, testes de CRUD e validação de estados duplicados.
Encomendas	Listagem de encomendas	Implementada, mas não validada	src/pages/painel/EncomendasLista.tsx	Sem pesquisa, filtros, ordenação, paginação ou ações funcionais; carrega todas as linhas.	Adicionar paginação, pesquisa e ações, com testes de RLS.
Encomendas	Criação de encomenda com itens	Bloqueada ou com problemas	src/pages/painel/EncomendaForm.tsx; encomendas.numero_encomenda NOT NULL em schema.sql	O formulário não envia numero_encomenda e a função gerar_numero_encomenda não é chamada. A criação falha.	Gerar o número no servidor (trigger/RPC transacional) e cobrir criação + itens numa transação.
Encomendas	Edição de encomenda	Bloqueada ou com problemas	src/pages/painel/EncomendaForm.tsx (fetchEncomenda vazio); rota /painel/encomendas/:id/editar	A rota existe, mas o formulário não carrega nem atualiza dados existentes.	Implementar leitura por id, atualização e gestão de itens com validação.
Encomendas	Atualização de estado com mensagem e fotos	Planeada	Tabelas historico_estados e fotos_encomenda; não existe UI de alteração de estado	Os botões da lista não têm handlers. O histórico nunca é criado pela aplicação.	Criar fluxo de atualização transacional, upload validado e registo automático no histórico.
Encomendas	Numeração sequencial de encomendas	Parcialmente implementada	função gerar_numero_encomenda em schema.sql; não é chamada pelo frontend	MAX+1 não é seguro sob concorrência e pode violar UNIQUE(empresa_id, numero_encomenda).	Usar sequência por empresa/ano ou transação com bloqueio; expor por RPC.
Documentos	Numeração de recibos	Parcialmente implementada	função gerar_numero_recibo em schema.sql; nenhuma UI de emissão	Sem restrição UNIQUE em numero_recibo; concorrência pode duplicar; não há emissão.	Criar constraint e fluxo de emissão transacional antes de gerar o PDF.
Documentos	Código de verificação de certificado	Parcialmente implementada	função gerar_codigo_verificacao em schema.sql; tabela certificados	Usa RANDOM(), não é criptográfico; não há retry em colisão; não existe UI de emissão.	Gerar com gen_random_bytes, tentar novamente em colisão e registar auditoria.
Portal público	Página de acompanhamento por token	Bloqueada ou com problemas	src/pages/publico/AcompanharEncomenda.tsx; RLS sem políticas para anon	A página consulta tabelas diretamente como anon. Sem políticas de leitura, a consulta falha.	Substituir a consulta pela RPC obter_encomenda_publica e tratar null/erro.
Portal público	RPC pública obter_encomenda_publica	Implementada, mas não validada	supabase/schema.sql (SECURITY DEFINER; GRANT EXECUTE TO anon)	Não é usada pela UI; sem SET search_path, sem testes, sem rate limiting; devolve IDs internos.	Endurecer a função, testar com token válido/inválido e ligar a UI.
Portal público	Timeline, estado atual e previsão de entrega	Parcialmente implementada	AcompanharEncomenda.tsx (UI); historico_estados	A UI existe, mas não recebe dados; a previsão nula produz data inválida/1970.	Ligar à RPC, tratar datas nulas e mostrar estados configurados.

Módulo	Funcionalidade	Estado	Evidência	Limitações	Próxima ação
Portal público	Fotos, recibo, certificado, avaliação e contacto	Planeada	Tabelas e RPCs existem; UI não implementa estas secções	A página atual não mostra fotos, recibos, certificados nem formulário de avaliação.	Implementar secções com dados da RPC e estados vazios.
Passaporte	Página pública da peça (/peca/:tokenPeca)	Bloqueada ou com problemas	src/pages/publico/PecaPassaporte.tsx; rota usa :tokenPeca e o componente lê token_peca	Página stub, sem fetch, com URL de modelo placeholder e parâmetro incompatível.	Corrigir o parâmetro e consumir obter_peca_publica.
Passaporte	Visualizador 3D GLB/GLTF	Parcialmente implementada	src/components/viewer3d/Viewer3D.tsx	Sem STL, sem fallback de erro, sem loading visível; não está integrado em página funcional.	Adicionar STLLoader, error boundary, fallback e testes com modelos reais.
Certificação	Verificação pública de certificado	Bloqueada ou com problemas	src/pages/publico/VerificarCertificado.tsx; RLS de certificados; RPC verificar_certificado	Consulta a tabela como anon (bloqueada), ignora o parâmetro :codigo e não mostra empresa/data/selo.	Usar a RPC, preencher o código a partir da rota e apresentar todos os campos.
Certificação	Emissão e template de certificado	Planeada	tabela certificados; pdfGenerators.ts (esboço)	Não existe fluxo de emissão nem editor de template.	Criar emissão, template por empresa, PDF com marca e QR.
Documentos	Recibo em PDF	Parcialmente implementada	src/lib/pdfGenerators.ts (gerarReciboPDF)	Esboço sem itens, marca, QR ou numeração; não é chamado por nenhuma página.	Implementar layout, dados reais e testes de geração.
Documentos	Certificado em PDF	Parcialmente implementada	src/lib/pdfGenerators.ts (gerarCertificadoPDF)	Esboço simples, sem marca, critérios, selo gráfico ou QR.	Implementar layout premium e verificação por QR.
QR	Geração de QR PNG/PDF e cartão de embalagem	Planeada	qrcode.react instalado; nenhuma importação no código	Não existe geração nem download.	Implementar QR do link público/passaporte e exportação PNG/PDF.
Avaliações	Submissão pública de avaliação	Planeada	RPC submeter_avaliacao em schema.sql; tabela avaliacoes	Não existe formulário na página pública; a RPC não valida o estado da encomenda nem tem rate limiting.	Criar UI, validar estrelas no servidor e limitar abuso.
Dashboard	Contagem total de encomendas	Implementada, mas não validada	src/pages/painel/Dashboard.tsx (count exact)	Depende de sessão/RLS; sem testes; não há filtro por período.	Validar com dados de teste e adicionar filtros.
Dashboard	Tempo médio de fabrico e avaliação média	Bloqueada ou com problemas	Dashboard.tsx valores fixos "3 dias" e "4.8"	Valores inventados/hardcoded, não calculados a partir da base de dados.	Remover ou calcular com queries agregadas e estados de carregamento.
Dashboard	Indicadores mensais e relatórios	Planeada	Não existe código de agregação mensal	Apenas cartões estáticos; sem gráficos nem exportação.	Definir métricas, queries e visualizações.
Encomendas	Pesquisa, filtros, ordenação e paginação	Planeada	EncomendasLista.tsx importa Search/Filter mas não os usa	A lista carrega tudo e não tem controlos.	Adicionar query params, paginação server-side e testes.
Segurança	Isolamento entre empresas por RLS	Implementada, mas não validada	Políticas RLS em schema.sql para 9 tabelas	Sem testes de isolamento; políticas de Storage não seguem o mesmo modelo.	Criar suite de testes RLS com dois utilizadores/empresas.
Segurança	Bucket privado e URLs assinadas	Bloqueada ou com problemas	schema.sql: buckets empresas e encomendas com public=true	Fotos de encomendas, ficheiros 3D e documentos ficam acessíveis por URL pública.	Tornar encomendas privado, migrar URLs e usar URLs assinadas.
Segurança	Validação de uploads (tipo e tamanho)	Bloqueada ou com problemas	utils.ts define validarImagem/validarFicheiro3D e limites; não são usados	Aceita qualquer ficheiro; sem limite no cliente nem no servidor.	Aplicar validação no cliente e políticas/limites no Storage.
Segurança	Funções RPC endurecidas	Parcialmente implementada	RPCs SECURITY DEFINER em schema.sql	Sem SET search_path, sem validação robusta, sem rate limiting; grants a anon.	Fixar search_path, validar entradas, limitar campos e monitorizar abuso.
Dados	Seed de demonstração	Bloqueada ou com problemas	supabase/seed.sql	Usa utilizador_id inexistente e não fornece numero_encomenda NOT NULL; falha.	Criar seed idempotente com utilizador real de teste e todos os campos obrigatórios.
Build	Build de produção	Bloqueada ou com problemas	npm run build e npx vite build falham (TS5101; @apply btn; erros TS)	Não é possível gerar dist para Vercel. Não há verificação de tipos verde.	Corrigir tsconfig/types e a CSS Tailwind, e adicionar CI que impeça regressões.
Qualidade	Testes automatizados	Planeada	Não existe framework nem ficheiros de teste	Nenhuma funcionalidade tem validação automatizada.	Introduzir Vitest/Testing Library e Playwright, começando por RLS e fluxos críticos.
Qualidade	Acessibilidade WCAG 2.2 AA	Não confirmada	Labels presentes em formulários; botões de ícone sem aria-label	Não foi feita auditoria; foco, contraste e leitores de ecrã não validados.	Auditar com axe/Lighthouse e corrigir semântica e foco.
Conformidade	RGPD, privacidade e termos	Planeada	Não existem páginas legais, política de retenção ou exportação/eliminação	Dados pessoais de compradores são tratados sem documentação de conformidade.	Nomear responsável, mapear tratamentos, criar políticas e procedimentos.

Módulo	Funcionalidade	Estado	Evidência	Limitações	Próxima ação
Operações	Deploy Vercel com rewrites SPA	Planeada	Não existe vercel.json; build falha	Rotas profundas como /acompanhar/:token devolveriam 404 sem rewrite.	Adicionar configuração, ambiente de preview e smoke tests.

Anexo B — Matriz de permissões

Recurso	Anon	Vendedor (dono)	Outro vendedor	service_role
Empresas	Nenhum	SELECT/UPDATE da sua	Nenhum	Total
Configuração de estados	Nenhum	CRUD da sua	Nenhum	Total
Encomendas	Nenhum	CRUD das suas	Nenhum	Total
Itens	Nenhum	CRUD dos seus	Nenhum	Total
Histórico	Nenhum	CRUD do seu	Nenhum	Total
Fotos	Nenhum	CRUD das suas	Nenhum	Total
Recibos	Nenhum	CRUD dos seus	Nenhum	Total
Certificados	Nenhum	CRUD dos seus	Nenhum	Total
Avaliações	RPC de submissão	SELECT das suas	Nenhum	Total
Storage empresas	Leitura	Escrita (sem restrição de pasta)	Escrita (falha de isolamento)	Total
Storage encomendas	Leitura	Escrita (sem restrição de pasta)	Escrita (falha de isolamento)	Total

Anexo C — Catálogo de requisitos

Os requisitos RF-001 a RF-030, RNF-001 a RNF-012 e SEG-001 a SEG-012 estão definidos no Capítulo 2. O catálogo deve ser atualizado sempre que um requisito for implementado ou validado, com ligação ao teste correspondente.

Anexo D — Referências oficiais

Assunto	Fonte	Consulta
React	https://react.dev/	27/10/2026
Vite	https://vite.dev/	27/10/2026
Tailwind CSS	https://tailwindcss.com/docs	27/10/2026
Supabase Auth	https://supabase.com/docs/guides/auth	27/10/2026
Supabase RLS	https://supabase.com/docs/guides/database/postgres/row-level-security	27/10/2026
PostgreSQL RLS	https://www.postgresql.org/docs/current/ddl-rowsecurity.html	27/10/2026
jsPDF	https://github.com/parallax/jsPDF	27/10/2026
three.js	https://threejs.org/docs/	27/10/2026
Vercel	https://vercel.com/docs	27/10/2026
CNPD	https://www.cnpd.pt/	27/10/2026
EDPB	https://www.edpb.europa.eu/	27/10/2026
Portal das Finanças	https://info.portaldasfinancas.gov.pt/	27/10/2026

As referências são indicadas para consulta futura. A sua inclusão não significa que o projeto cumpra os requisitos nelas descritos.

Anexo E — Dependências relevantes

Dependência	Versão	Uso
react / react-dom	19.3.1	UI
vite	8.3.4	Build
typescript	~6.0.3	Tipos
tailwindcss	4.3.3	Estilos
react-router-dom	7.18.4	Rotas
@supabase/supabase-js	2.118.0	Backend
three / @react-three/fiber / drei	0.186.1 / 9.8.2 / 10.7.9	3D
jspdf / jspdf-autotable	4.2.1 / 5.0.9	PDF
qrcode.react	4.0.1	QR (não usado)
date-fns	4.4.0	Datas
lucide-react	0.468.0	Ícones
oxlint	1.88.0	Lint

Anexo F — Histórico da documentação

Versão	Data	Alteração	Autor
1.0	27/10/2026	Auditoria inicial, identidade visual, apresentação, documentação e manual.	Equipa OrbitTrack

Anexo G — Pontos por validar

01 Execução real do schema num projeto Supabase limpo, incluindo pgcrypto.
02 Criação automática da empresa após registo.
03 Confirmação de email e redirect URLs.
04 Isolamento RLS entre duas empresas.
05 Políticas de Storage e acesso anónimo.
06 Criação de encomendas após correção da numeração.
07 Link público e verificação de certificados com as RPCs.
08 Formato e conteúdo dos PDFs.
09 Compatibilidade dos modelos 3D e STL.
10 Acessibilidade e desempenho em dispositivos reais.
11 Requisitos legais e fiscais com revisão especializada.

Anexo H — Índice de evidências

Evidência	Localização	Conteúdo
Esquema	supabase/schema.sql	Tabelas, funções, triggers, RLS e Storage.
Seed	supabase/seed.sql	Dados de demonstração (inválido).

Evidência	Localização	Conteúdo
Rotas	src/App.tsx	Mapa de navegação.
Autenticação	src/contexts/AuthContext.tsx	Sessão e operações de conta.
Encomendas	src/pages/painel/EncomendaForm.tsx	Criação e edição.
Portal	src/pages/publico/AcompanharEncomenda.tsx	Interface do comprador.
Verificação	src/pages/publico/VerificarCertificado.tsx	Verificação de certificados.
PDF	src/lib/pdfGenerators.ts	Esboços de documentos.
Build	package.json / tsconfig.json / src/index.css	Comandos e erros.
Apresentação	entrega/OrbitTrack_Apresentacao_Comercial.pptx	Deck comercial.

> **Nota final**
>
> Esta documentação descreve o estado do código na data indicada. Qualquer alteração ao schema, às rotas ou às políticas de segurança invalida as conclusões e exige nova auditoria.